

A neural symbolic model for space physics

Received: 12 April 2025

Accepted: 4 September 2025

Published online: 15 October 2025



Jie Ying ^{1,11}, Haowei Lin ^{2,11}, Chao Yue ^{3,11}, Yajie Chen ⁴, Chao Xiao ⁵,
Quanqi Shi ⁵, Yitao Liang ², Shing-Tung Yau ^{6,7}✉, Yuan Zhou ^{6,7,8}✉ &
Jianzhu Ma ^{9,10}✉

Symbolic regression, a key problem in discovering physics formulas from observational data, faces persistent challenges in scalability and interpretability. We introduce PhyE2E, an AI framework designed to discover physically meaningful symbolic expressions. PhyE2E decomposes the symbolic regression problem into subproblems via second-order neural network derivatives, and employs a transformer architecture to translate data into symbolic formulas in an end-to-end manner. The generated expressions are further refined via Monte Carlo tree search and genetic programming. We leverage a large language model to synthesize extensive expressions resembling real physics, and train the model to recover these formulas directly from data. Comprehensive evaluations demonstrate that PhyE2E outperforms existing state-of-the-art approaches, delivering superior symbolic accuracy, fitting precision and unit consistency. We deployed PhyE2E on five critical applications in space physics. The AI-derived formulas exhibit excellent agreement with empirical data from satellites and astronomical telescopes. We improved NASA's 1993 formula for solar activity and provided an explicit symbolic explanation of the long-term solar cycle. We also found that the decay of near-Earth plasma pressure is proportional to the square of the distance r from the Earth's centre, with subsequent mathematical derivations validated by independent satellite observations. Furthermore, we found symbolic formulas relating solar extreme ultraviolet emission lines to temperature, electron density and magnetic-field variations. The formulas obtained are consistent with properties previously hypothesized by physicists.

The primary distinction between physics and other data sciences is the pursuit of the discovery of fundamental laws behind the world using concise symbolic formulas. The discovery of physics formulas is a lengthy process of trial and error. For instance, after about 40 attempts to match Mars data with various elliptical shapes, Kepler discovered that Mars' orbit was elliptical and proposed his three laws¹. Similarly, through meticulous experimentation on electric and magnetic phenomena, Faraday unveiled the laws of electromagnetic induction. He demonstrated that the intricate relationship between electricity and magnetism could be articulated through elegant, fundamental principles derived from empirical data².

Discovering laws in physics or other specialized fields often demands years of domain expertise. Numerous methods have been specifically developed and proposed to aid in the discovery of physics formulas and to enhance the understanding of the underlying principles^{3–5}. Since AI has achieved tremendous success in multiple domains, a naturally arising question is whether we can leverage AI to automatically extract physical laws from experimental observation data to better understand our physical world. This task is known as symbolic regression within the AI field and has received widespread attention in recent years. Genetic algorithms (GAs) were first adopted to address symbolic regression problems by evolving a population of

A full list of affiliations appears at the end of the paper. ✉e-mail: styau@tsinghua.edu.cn; yuan-zhou@tsinghua.edu.cn; majianzhu@tsinghua.edu.cn

candidate symbolic expressions (typically represented as trees) to minimize a fitness function, which measures how well the expression fits the data^{6–15}. Monte Carlo tree search (MCTS) predicts the expression by exploring a search tree of possible expressions, simulating paths down the tree by randomly selecting mathematical operations to extend expressions. For each path, it generates complete expressions and assesses fitness on the basis of how well they approximate the target function^{16–20}. Compared with GAs, MCTS offers more dynamic and fine-grained control over reward signals. Rewards can be customized to reflect the quality of partial solutions during tree exploration, facilitating a more efficient search for the optimal solution. Recent advances in deep neural networks have paved the way for the development of end-to-end methodologies. Unlike GAs and MCTS, which are symbolic search techniques that frequently struggle with the expansive search space, the end-to-end approach streamlines the process by eliminating the need for iterative searching and refinement. It predicts mathematical expressions as sequences of symbols (operators, variables, constants) from data, allowing symbolic expressions to be generated in a single forward pass through the neural network, which substantially improves speed, especially for large datasets and complex functions^{21–26}. End-to-end approaches require large datasets for training, yet the number of physical formulas discovered by humanity remains relatively limited. There are still many unknown areas in the physical world waiting for humanity to discover and formulate new laws and equations. Therefore, current data-driven end-to-end algorithms are limited in reasoning about simple mathematical equations and have not been applied to real physical data. Additionally, symbols in physical law have physical unit systems, and the entire expression should ensure the correctness of these units, which is not yet considered by current computational methods.

To address these limitations, we propose a new framework, named PhyE2E, to achieve accurate symbolic regression for space physics. The PhyE2E framework includes the following key components. First, we fine-tuned a large language model (LLM) with existing physics formulas, enabling it to generate a diverse array of formulas that align with the statistical distribution of physics formulas. By harnessing the common knowledge acquired by LLMs from the Internet, the generative model efficiently learns the underlying distribution from a small set of seed formulas, thereby overcoming the challenge of data sparsity in the training process. Second, we trained an end-to-end formula regression model based on the transformer model, which directly converts data matrices into symbolic formulas encoded in Polish representation^{24–26}. With the aid of the LLM, our end-to-end model is trained with a large volume of formulas that ‘look physical’ and adhere to consistent unit systems. Third, to reduce the search complexity, we developed a formula-splitting technique that is able to group variables without nonlinear (or logarithmically nonlinear) relationships, producing a series of simpler subformulas for a nested and more simplified symbolic regression task. This technique uses an oracle neural network to fit the data and then analyses its second-order derivatives to uncover relationships among the input variables. Finally, we leveraged the state-of-the-art GA and MCTS methods to further refine the predicted formulas. To evaluate our performance, we conducted comparisons on both an LLM-synthesized dataset and another real-world physics dataset, AI Feynman, against GA-based, transformer-based and neural network (NN)-based methods. Experimental results indicate that our method outperforms all others in terms of data-fitting accuracy, the correctness of the mathematical form of the formulas, and unit accuracy. We further applied our model to various important applications in space physics, including predicting sunspot numbers (SSNs), plasma-sheet pressure, solar rotational angular velocity, emission-line contribution functions from the Sun, and lunar tidal signals in the plasma layer. Compared with formulas proposed by physicists, the formulas derived from PhyE2E exhibit better generalizability, a more concise mathematical form and more precise physical units, and more

importantly provide physically meaningful insights and explanations for instrumental observations.

Results

Overview of PhyE2E

PhyE2E comprises an end-to-end transformer-based physical model designed to take observed data points as input and predict both the operators and the physical units involved in a formula directly (Fig. 1). A set of 264,000 synthetic formulas was generated by an LLM (OpenL-LaMA2-3B) that had been fine-tuned using real physics formulas from the Feynman Symbolic Regression Database²⁷ (Fig. 1 top). We randomly selected 180,000 formulas from the synthetic dataset for training, reserving the remaining data for testing. The inference process of PhyE2E involves three stages: a variable-interaction detection method to decompose the target formula into subformulas (Fig. 1 bottom left), end-to-end prediction of each subformula using the trained model (Fig. 1 middle) and GA and MCTS approaches to refine the predicted formulas (Fig. 1 bottom right). More specifically, we first fitted the data using a standard multilayer neural network, and used its Hessian matrix to determine the nonlinear interactions between all pairs of variables (Methods). The core of our model leverages a transformer architecture, synthesizing formulas as a prefix sequence with attached physical units (Methods). This approach incorporates both observed data points and physical prior knowledge about the target formula. In the final stage, we utilized GA and MCTS approaches to minimize the root mean squared error of the predicted formula (Methods). This was achieved by using a grammar pool of context-free rules that incorporates basic operators such as exp, as well as the most promising subformulas, which were automatically constructed within the search tree by PhyE2E.

Performance on the synthetic and AI Feynman datasets

We divided the synthetic dataset containing 264,000 formulas into training, validation and test sets with a ratio of 80%, 10% and 10%. We ensure that all validation and test formulas are unseen during training by removing formulas that are either identical, or become completely equivalent to the formulas in the training set after simple mathematical transformations (Methods). We compared PhyE2E with 15 state-of-the-art symbolic regression baseline models, including four GA-based models (PySR²⁸, GP-GOMEA¹⁰, Operon²⁹, gplearn¹¹), three transformer-based models (TPSR²⁶, EndToEnd²⁴, NeSymReS²⁵), two LLM-based models (LaSR³⁰, LLM-SR³¹) and six NN-based models (uDSR³², PhySO³³, AI Feynman^{27,34}, ParFam³⁵, KAN^{36,37}, BSR³⁸). The technical details of running these models are provided in Supplementary Note 3. The detailed comparisons with PySR under different configurations are provided in Supplementary Note 4. We also included two variants of PhyE2E in our comparison, including a version without the formula decomposition module (divide and conquer, D&C) or the MCTS refinement module (MCTS). We evaluated the performance of all the models on six metrics³⁹ including symbolic accuracy, average accuracy ($R^2 > 0.99$), unit accuracy, complexity, complexity relative to the ground truth formulas and elapsed times (Methods).

First, we evaluated whether the physics formulas synthesized by the LLM were consistent with the real physics formulas from the Feynman Symbolic Regression Database. It can be observed that the formulas generated by the LLM closely match the distribution of real physics formulas in terms of the number of variables, formula complexity, depth and operator types measured using the Jensen–Shannon divergence (D_{JS}) (Methods and Fig. 2a). Then, we studied the symbolic accuracy of the formulas generated by the model, that is, whether the mathematical forms of the formulas correspond to the true formulas used to generate the data. PhyE2E(D&C + MCTS) exceeds the second-best model, PySR, by 26.48% and outperforms the best NN-based model, uDSR, by 31.75%, the best LLM-based model, LaSR, by 37.89% and the best transformer-based model, TPSR, by 39.83% (Fig. 2b). The formulas generated by PhyE2E also demonstrate

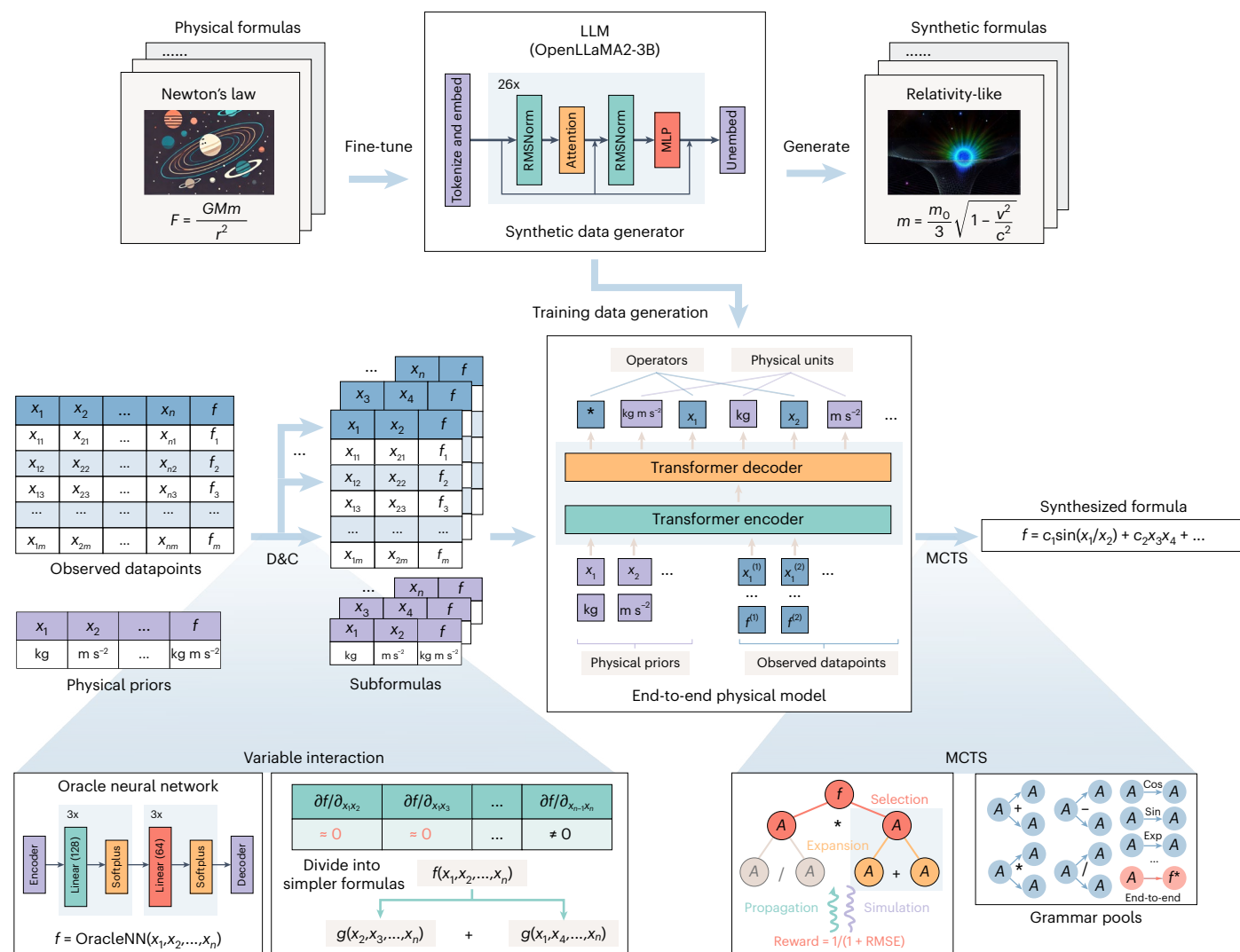


Fig. 1 | The overall PhyE2E framework. Top: the training dataset was augmented with a large-scale synthetic dataset generated by an LLM. MLP, multilayer perceptron. Middle: a variable-interaction technique was integrated to decompose the original symbolic regression problem into simpler subproblems, referred to as D&C. An end-to-end model was trained to predict the target

formula using observed data points and prior physical knowledge (referred to as 'physical priors'). Bottom: an MCTS module was adopted to refine the generated formulas, using a context-free grammar pool that includes atomic formulas and the end-to-end generated formula. RMSE, root mean squared error.

a more powerful ability to fit data compared with other methods. PhyE2E (D&C + MCTS) outperforms all the state-of-the-art approaches by at least 20.00% in terms of average accuracy: Avg.Acc. ($R^2 > 0.99$). Regarding the accuracy of (physical) units, we observe that PhyE2E achieves 99.27% accuracy, leading all the other approaches. The performance drops to 93.30% on including the D&C and MCTS modules. This decline is due to the absence of unit constraints during the D&C and MCTS refinement stage. The most compatible baseline of unit accuracy is PhySO, which reaches a comparable 89.70% with a strict unit constraint during its search process³³.

The data-fitting capabilities of the formulas can be enhanced by increasing their complexity. According to Occam's razor, complex formulas tend to have weaker generalizations compared with simpler ones and lose their interpretability in a physical context. Therefore, in addition to studying the formulas' ability to fit data, we also focus on the complexity of the formulas generated by different models. Our model produces formulas with lower model complexity compared with the formulas generated by other models. For 42.17% of the test formulas, their depth is less than 3, suggesting that they predominantly represent linear relationships. For these low-complexity formulas, our

model successfully recovers the mathematical form of 98.02% of them (Fig. 2d). The complexity relative to the ground-truth formula of PhyE2E is 2, which is 33.27% better than that of the second-best model, PySR. On introducing the D&C and MCTS modules, PhyE2E increases the complexity of formulas by 6.79%, while still maintaining a complexity similar to that of PySR. Only a handful of other baseline models demonstrate the capability to predict formulas of appropriate complexity (for example, PySR, GP-GOMEA, AI Feynman, PhySO), primarily because of their constraints on complexity or the inclusion of physical units. Others generate either formulas with high complexity (>50) or formulas that deviate far away from the target formula (relative complexity >10), making them difficult to interpret in practice (Fig. 2b).

Next, we evaluated the performance of different models on the formulas collected from the Feynman Symbolic Regression Database, referred to as the Feynman dataset for brevity. The Feynman dataset contains 100 real-world physics formulas sampled from the seminal Feynman Lectures on Physics^{40–42}, covering core physics topics such as classical mechanics, electromagnetism and quantum mechanics. Although our model was not directly trained on formulas from the Feynman dataset, our training dataset was generated using an LLM on

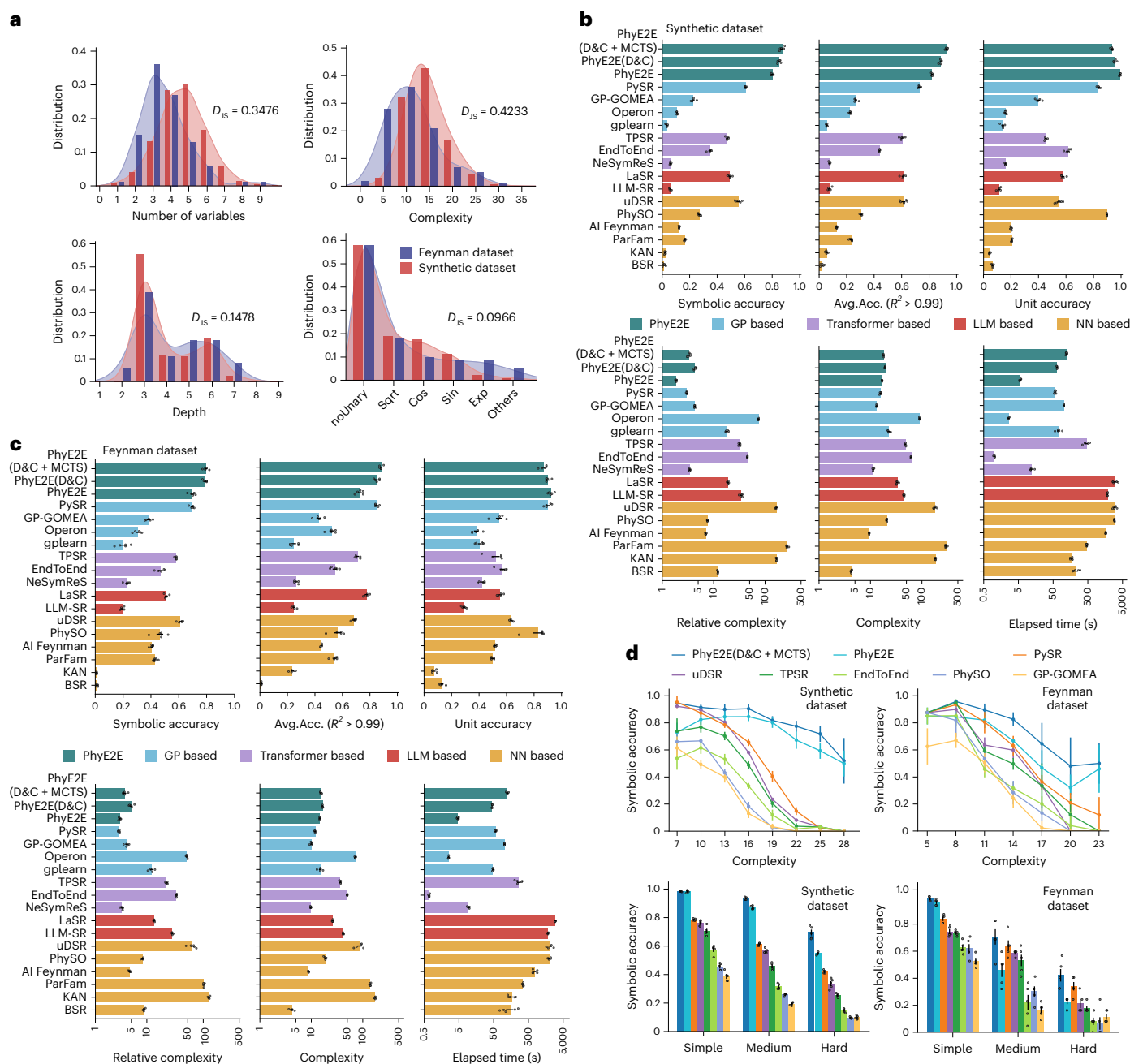


Fig. 2 | Performance on the synthetic and AI Feynman datasets. a, Comparison between the formulas generated from LLaMA2 and the Feynman dataset. The distance between the distributions of different properties of the two sets of formulas is measured using D_{JS} . noUnary, formulas consisting solely of binary operators and excluding any unary operators. **b, c**, Evaluation results for symbolic regression methods on the test set of the synthetic dataset

(b) and AI Feynman dataset **(c)**, respectively. Data are presented as mean $\pm$ s.e.m. ($n = 5$ individual trials for each baseline). **d**, Evaluation results for formulas with different complexities (upper panels) and different difficulties (bottom panels) on the synthetic and AI Feynman datasets. The bar plots represent mean $\pm$ s.e.m. ($n = 5$ individual trials for each baseline).

the basis of formulas from the Feynman dataset. To inhibit data leakage, we removed the formulas in the training dataset that were either identical, or became completely equivalent after simple mathematical transformations to those in the Feynman dataset. A comprehensive list of the test formulas is provided in Supplementary Table 1.

First, we observe that all computational methods, including ours, show similar performances on the Feynman dataset and on our synthetic data, which demonstrates that the distribution of formulas generated by the LLM is essentially consistent with those from the Feynman dataset. In terms of symbolic accuracy, PhyE2E(D&C + MCTS) exceeds the best classical model, PySR, by 10.09%, the best NN-based

model, uDSR, by 18.49%, the best transformer-based model, TPSR, by 21.77% and the best LLM-based model, LaSR, by 29.35%. Although PySR outperforms standard PhyE2E in numerical precision ($R^2 > 0.99$), with an accuracy of 84.96%, it is still surpassed by PhyE2E with the D&C and MCTS modules. The complexity of the models generated by PhyE2E remains low. The complexity of PhyE2E relative to the ground-truth formula is 2.98, 3.67% higher than that of the best classical model, PySR. The relative complexity for PhyE2E is lower than 5.75 and the complexity is lower than 16.85, which is essentially the best among all the baseline models (Fig. 2c). To further investigate the relationship between performance and formula complexity, we calculated

the symbolic accuracy as a function of the number of variables, the complexity and the number of unary and binary operations. We found that our method had a clear advantage over other methods for formulas with high complexity. When the complexity is larger than 20, our model outperformed the second-best methods by 67.11% and 32.73% on the two datasets, respectively (Fig. 2d). We divided both datasets into three subsets of varying difficulty on the basis of the similarity of mathematical formulas to those in training datasets (0.95–1.0 as easy, 0.80–0.95 as medium, 0.00–0.80 as hard), and then systematically calculated the symbolic accuracy of each method. We found that our method had a notable advantage on the medium and hard datasets (Methods and Fig. 2d).

Among all the modules, the D&C module plays a crucial role in simplifying the search space in our framework. Consider a formula $f = m\sqrt{B_1^2 + B_2^2 + B_3^2}$ from the Feynman dataset; our D&C module first determined that the three variables B_1 , B_2 and B_3 under the square root do not interact, which indicates that the operators between them can only be addition or subtraction (Extended Data Fig. 1a). Therefore, the original symbolic regression task decomposes into three parts, $g_i = m\sqrt{B_i^2 + C_i}$ ($i = 1, 2, 3$), each of which is processed individually by the end-to-end module. The predicted formulas for g_1 , g_2 and g_3 are later aggregated into one complete formula (Extended Data Fig. 1a). The D&C module reduces the complexity of a formula by splitting a set of variables that have no nonlinear interactions locally within a certain formula. This also explains why we find that our method has a considerable advantage over other methods as the complexity of the formulas increases.

However, the risk associated with this methodology is that if the decomposition is incorrect some of the segments will contain incorrect variables. Therefore, we carefully studied the performance of different D&C strategies. We implemented four different strategies: single pattern (detects additive and multiplicative interaction), multipattern (identifies all interactions such as additive terms under sine functions), fixed threshold and adaptive threshold (Methods). We evaluated the performance of different strategies by assessing how many formulas could be correctly decomposed, partially correctly decomposed and completely incorrectly decomposed on the test set of the synthetic dataset. We found that the adaptive threshold strategy provided a flexible approach for interaction detection, resulting in a substantial improvement in complete accuracy by 50.61%. The multipattern strategy facilitates diverse types of interaction and effectively reduces the proportion of absolutely wrong formulas from 6.50% to 5.03% (Extended Data Fig. 1b). Overall, adopting the multipattern and adaptive threshold strategies results in the highest number of accurate formula decompositions; this was the strategy we used in our framework.

To discover physics formulas rather than purely mathematics formulas, another important technique is the consistent units of physics quantities^{33,34}. We further retrain two additional models using the same architecture but one model without a unit decoding strategy and another without any physical priors, thereby excluding the unit decoding strategy as well (Methods). These three models are evaluated using three accuracy metrics on the synthetic dataset (Extended Data Fig. 1d). We found that the unit prior plays an important role, especially when dealing with a small number of input data. In an extreme case with only five input data points, the symbolic accuracy of PhyE2E improved to 39.83%, compared with 26.06% without a unit decoding strategy and 8.97% without any physical priors. Another observation is that the unit accuracy is notably enhanced by the incorporation of physical priors. The unit accuracy improved by 25.90% compared with the PhyE2E without a unit decoding strategy, and by 56.70% compared with the PhyE2E without any physical priors in the five-data-point case. This improvement is also observed in the case with 50 data points, where the unit accuracy increased by 4.47% and 12.23%, respectively.

Performance of SSN prediction

Next, we applied our trained model to multiple applications in space physics. Our goal is to find formulas that are more accurate in prediction and simpler in mathematical form than existing physics formulas. We directly applied our trained model to these real-world applications instead of performing any fine-tuning operations. We started by studying the pattern of changes in the SSN over time. The Sun serves as the primary source of energy for the entire Earth system. Predicting SSNs is essential for forecasting space weather events that can impact both satellite operations and terrestrial communication systems. These forecasts are also pivotal in climate studies, aiding in modelling the impact of solar variability on the Earth's climate. The accurate prediction of sunspot activity is also crucial for managing the effects of geomagnetic storms on the technological infrastructure, which can lead to considerable disruptions and damage. Although the 11-yr solar cycle has been determined through direct observations of SSNs for the past four centuries, scientists want to know whether there is a longer cycle in solar activity that could influence the climate of Earth (Fig. 3a). Therefore, in this task, in addition to predicting the SSN, we also focus on how to derive the long-term cyclical variations of solar activity from the predicted physics formulas.

We first collected the SSN data from Sunspot Index and Long-term Solar Observations (SILSO) over the last 400 yr (ref. 43). The most widely adopted formula is the one proposed by Hathaway et al.⁴⁴, which was still being used in recent studies to analyse data from the last 30 yr (ref. 45). This formula modelled the SSNs $R(t)$ using different sets of parameters for different cycles (Fig. 3b). Although it can accurately fit the data for each cycle, it cannot be generalized from one cycle to another, making it incapable of predicting future SSNs or revealing the longer cycle of solar activity. To summarize a symbolic formula for SSN, we selected the SSN data from 1855 to 1976, which contain 11 cycles and 1,450 data points, as the input of our model. The main components of the denominator in our formula are a squared sine term and a squared cosine term, while the numerator consists of a squared sine term (Fig. 3c top). The Pearson correlation between the predicted SSN and the measured ones for the next four complete cycles (years 1976 to 2019) reaches 0.72. For the upcoming cycle (2019–present), PhyE2E predicts the peak value to be 177.40 and to occur on 10 October 2024 (Fig. 3c top). It is worth noting that no data were used to fine-tune or retrain our model. We did not use all the data to generate the formula because we needed to examine the generalizability of the formula obtained by the model. The generalizability of the model and the generalizability of the formulas predicted by our model should be evaluated separately. We tried generating formulas with different amounts of data, and the resulting formula forms remained largely consistent (Supplementary Tables 7 and 8).

We compared the formula generated by PhyE2E with those generated by other symbolic regression models. We retrained all the other models to be compared on data from 1855 to 1976 and tested their performance on data after 1976 to fairly compare with our model. We first examined the formulas generated by different symbolic regression models. We observed that, except for AI Feynman and GP-GOMEA, all methods produced formulas containing trigonometric functions capable of generating periodic outputs. Formulas from BSR, End-ToEnd, KAN, NeSymReS, PhySO and TPSR all yield identical values for each cycle, which clearly contradicts our observations and common sense. The formulas generated by gplearn, Operon, ParFam, uDSR and our method are capable of generating periodic outputs with different values for each cycle (Fig. 3c bottom). We further examined the formulas generated by PySR under different operator sets and different constraint variants (Supplementary Tables 11 and 12), and took the best PySR model with the highest avg-R score for comparison. However, the five other models generated excessively complex formulas, making it impractical to parse their physical meaning or ascertain the long-term cyclical patterns of solar activity (Supplementary Tables 9 and 10).

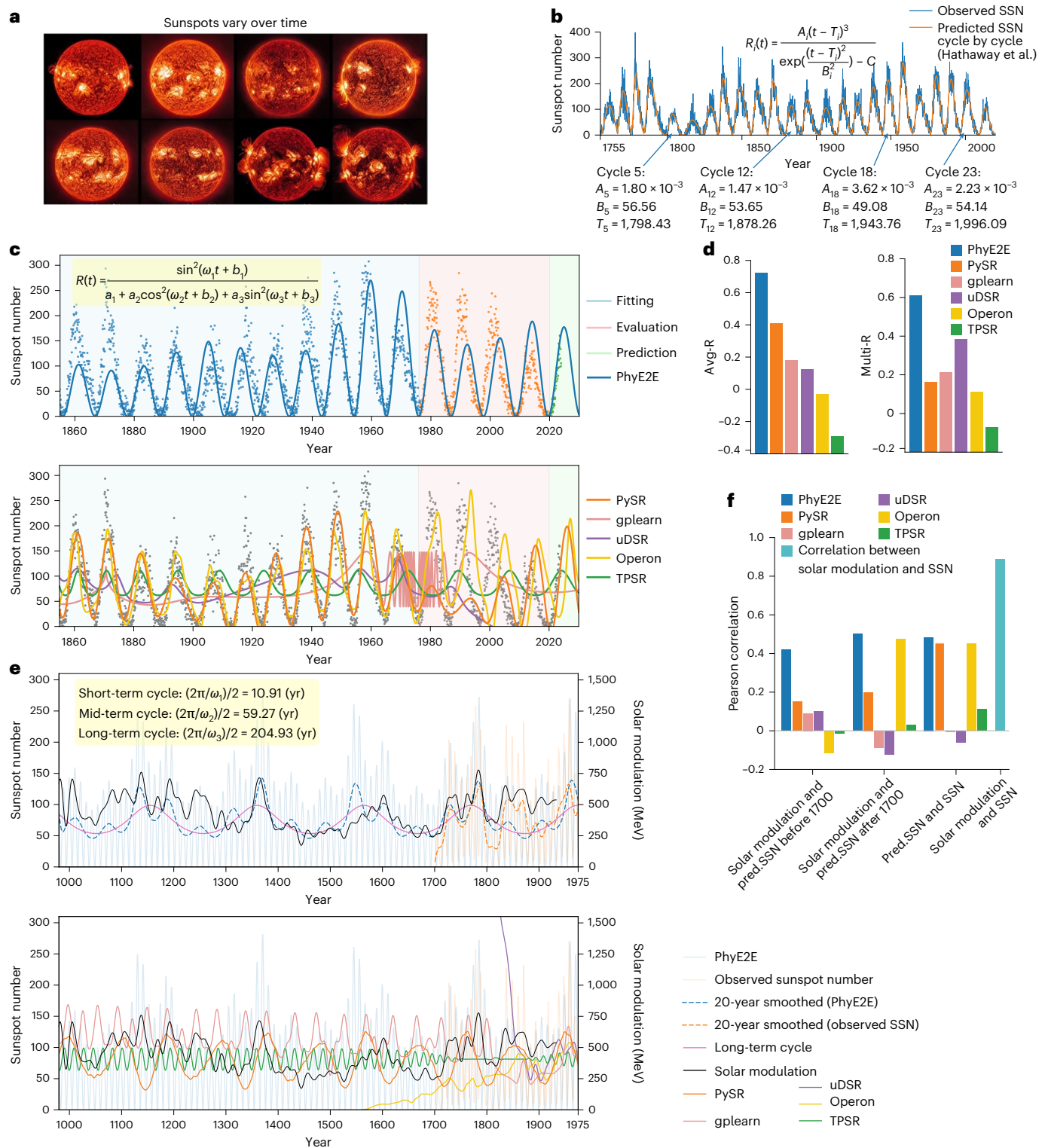


Fig. 3 | Performance of sunspot intensity prediction. **a**, Sunspot variation over time. **b**, Variations in SSN observed through telescopes from 1755 to 2020 and the formula derived by Hathaway et al.⁴⁴. **c**, Top: the PhyE2E formula and the variations in SSN yielded by the formula from 1855 to 1976. Bottom: the formulas generated by other baseline models and the variations in SSN yielded by these formulas from

1855 to 1976. **d**, Avg-R (left) and multi-R (right) on the test data from 1976 to 2019 for different baseline models. **e**, Solar modulation level and smoothed SSN from different baseline models over a longer time frame, from 980 to 1976. **f**, Pearson correlation between the SSN observed by telescopes, SSN predicted by the generated formulas, and solar modulation level from 980 to 1932.

Next, we quantified the performance of the formulas generated by these models on the test sets from 1976 to 2019 using two metrics: (1) the correlation between the formula's predictions and the measured data within each individual cycle, then these correlations averaged across multiple cycles (avg-R); (2) the correlation across multiple cycles (multi-R). Our simpler formulas substantially outperformed these complex formulas for both metrics. Specifically, the avg-R and multi-R of our method outperform the second-best methods by at least 76.01% and 58.57%, respectively (Fig. 3d).

To further validate the accuracy of the formula we obtained, we focused on the SSN data before the 1700s. Due to technological limitations, there are no SSN data directly observed from telescopes before 1749⁴³. Therefore, we collected solar modulation levels reconstructed from atmospheric ¹⁴C concentrations from the annual rings of thousand-year-old trees as an approximation of the SSN measurements⁴⁶. We adopted the same smoothing strategy as reported in ref. 46 and found that the Pearson correlation between solar modulation from tree rings and the SSN measured by the telescopes is 0.886 after the 1700s, which verifies their close relationship (Fig. 3f). Then, we compared the SSN generated by our formula and solar modulation data before the 1700s. Their Pearson correlation is 0.561 before 1300, 0.653 from 1300 to 1700 and 0.501 after 1700 (Fig. 3e), which further demonstrates the predictive capacity of our formulas on previously unseen data. Finally, since there are three sine/cosine functions in our formula, it is natural for us to derive three cycles from these trigonometric functions. The shortest cycle is 10.91 yr, which aligns with observations of solar activity widely accepted by the research community. The second longest cycle is 59.27 yr, which coincides exactly with the 60-yr cycle of the ancient Chinese astronomical calendar system of Heavenly Stems and Earthly Branches. The longest cycle is 204.93 yr, which we speculate is the cycle of the solar system operating within a larger planetary system (Fig. 3e). However, as this result is directly derived from symbolic formulas, it requires further confirmation through additional astronomical observational data. The constants of our formula are given in Supplementary Table 2.

Performance of plasma pressure prediction

Plasma pressure is a macroscopic parameter that plays an important role in plasma dynamics and the generation of electric currents. Increasing plasma pressure gradients in the radial direction causes stretching of magnetic field lines and enhances perpendicular currents flowing azimuthally. The azimuthal plasma pressure gradient generates field-aligned currents, resulting in bending of magnetic field lines (Fig. 4a). Wang et al. proposed a formula that describes how equatorial plasma pressure varies with its position relative to Earth using Geotail and the NASA mission Time History of Events and Macroscale Interactions during Substorms data. However, their formula involves exponential terms and nine constants for nightside equatorial isotropic plasma pressure, making it difficult to derive meaningful physical interpretations^{47,48} (Fig. 4b). To derive a simpler formula, we divided the same equatorial plasma pressure data according to the azimuthal angle, using the data from the near side of the Earth as input for our model and the data from the far side to assess the performance of our formula. As we increased the number of input data points, feeding the PhyE2E model with progressively farther data from Earth, the average mean squared error (MSE) decreased rapidly (Fig. 4c and Supplementary Tables 13 and 14). PhyE2E achieves an average MSE of 7.04×10^{-3} with only 10% of the data provided, demonstrating strong generalization capabilities with the small dataset as input. As more data were provided, the accuracy of our model continued to improve, eventually reaching 6.63×10^{-4} , resulting in more precise predictions for the regions on the far side of Earth (Fig. 4e). PhyE2E also outperforms all other baseline models in terms of fitting accuracy (MSE) and model complexity, delivering the most accurate and simplest prediction formula, while other models are unable to provide accurate predictions in certain areas. The formula

by Wang et al. cannot accurately predict the plasma pressure for the regions on the far side of Earth, while the EndToEnd method fails to provide accurate predictions for the regions on the near side of Earth (Fig. 4d). Note that this problem involves two variables, so there are two possible scenarios: one where the distance r from the Earth's centre and the azimuthal angle φ can be decomposed into two subformulas, and another where decomposition is not possible. We generated one formula for each scenario, compared their MSEs on the training dataset and selected the formula with the lower MSE as the final prediction. In this case, the formula derived using the decomposition method outperformed the alternative. The formula we predicted has a more concise mathematical form compared with the formula proposed by Wang et al. (Fig. 4b). It reveals to us that the decay of the near-Earth plasma pressure is proportional to r^2 , whereas in the formula proposed by Wang et al. the plasma pressure has an exponential relationship with r . More importantly, we can derive certain physical facts that align with the observational data from this new formula. Specifically, if the plasma pressure decays with the square of r and it is also known that the magnetic pressure decays with the sixth power of r , then, according to the formula plasma beta = magnetic pressure (P_{th})/plasma pressure (P_{p}), we can infer that plasma beta increases with the fourth power of r , which can be confirmed by observational data from another independent study⁴⁹. Among the formulas obtained by the other methods, only the EndToEnd approach²⁴ produced a formula that is inversely proportional to the square of r . However, its mathematical form is more complex compared with our formula (Supplementary Table 13). The constants of our formula are given in Supplementary Table 3.

Performance of solar rotational angular velocity prediction

The Sun's magnetic field is generated by the plasma motion within its interior. The angular velocity of solar rotation varies at different latitudes, and the magnetic field lines are stretched and twisted (Fig. 4f). Differential rotation is an important factor in the solar cycle for the prediction of magnetic fields and sunspots. Understanding solar differential rotation helps to improve the prediction of solar activities, which is important for predicting and mitigating the impact of space weather events on satellites and human activities in space. Differential rotation also provides key insights into the structure and dynamics of the solar interior by comparing rotation speeds at different latitudes with those predicted by comprehensive numerical models^{50,51}. One of the most widely adopted formulas describing the relationship between the solar differential rotation and solar latitude was derived by Snodgrass in 'Magnetic rotation of the solar photosphere'⁵². In that work, the author assumed that solar differential rotation was symmetric with respect to the equator. Such an assumption often fails, especially during periods of high levels of solar activity. In addition, this formula was fitted by using the measurements at low latitudes of the Sun, but observations at high latitudes are still missing, which limits the suitability of the model near the solar poles. For this task, the challenge for other AI models is that the limited number of data makes it impossible for them to train the model and predict the formulas.

PhyE2E derived a simpler and more accurate formula with a simple trigonometric term using the data from Snodgrass⁵². The largest difference between this formula and the one generated by PhyE2E lies in the difference in the trigonometric periodicity, leading to a steeper prediction for polar regions, rather than a flat one (Fig. 4g). We further reduced the number of training data points and found that PhyE2E could predict the same formula with only 14 data points as input, rapidly achieving an MSE of 1.31×10^{-4} , which outperforms other models (Fig. 4g and Supplementary Tables 15 and 16). In contrast to the oscillations seen in other models, PhyE2E exhibits exceptional consistency and robustness, providing stable predictions with varying numbers of input data. The constants of our model can be found in Supplementary Table 4. The predictions for all the baseline models are quite similar in non-polar regions, but they start to diverge in the

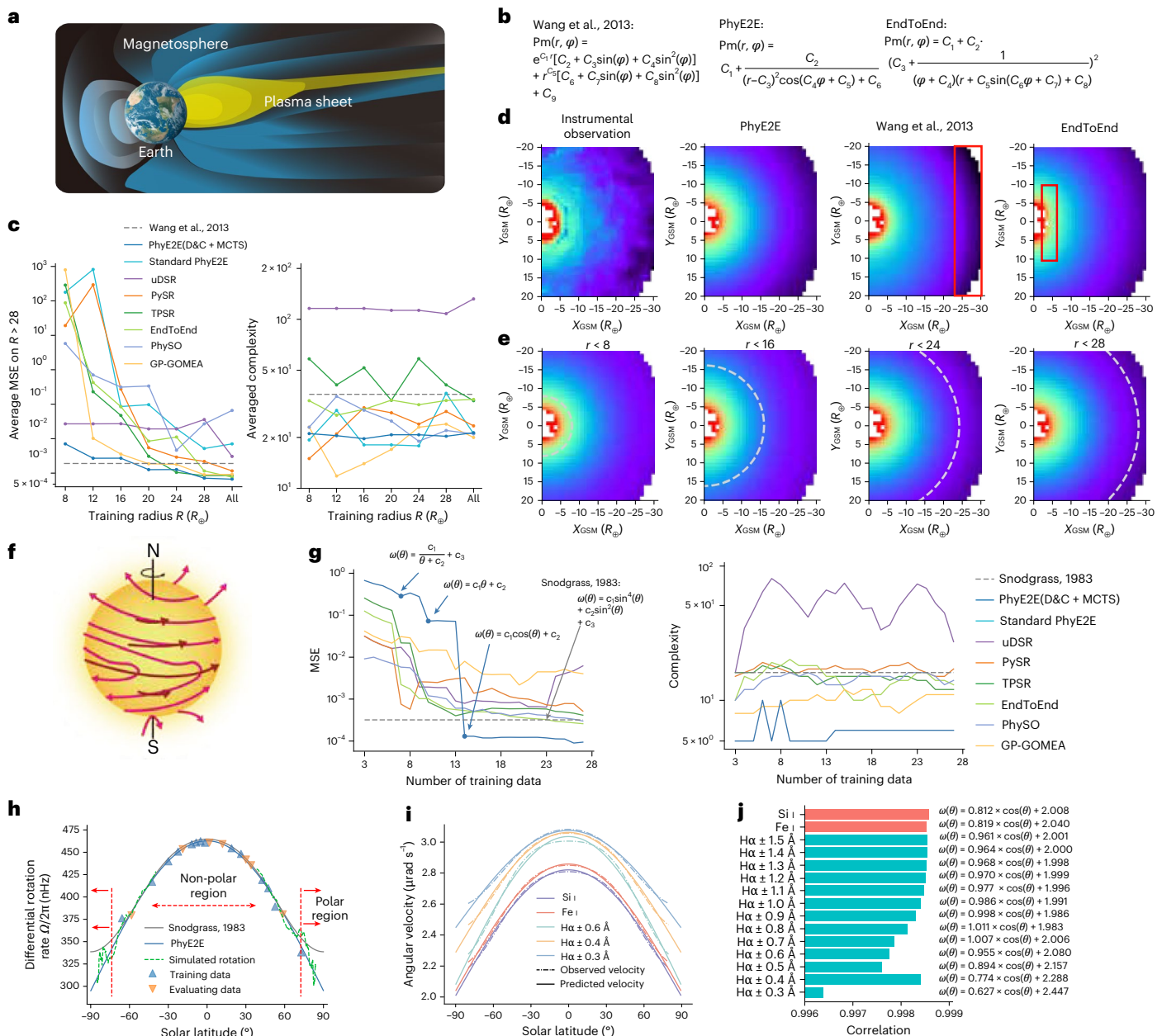


Fig. 4 | Performance of plasma-sheet pressure prediction and solar differential rotation prediction. a, Distribution of near-Earth magnetosphere and plasma sheet. **b**, Symbolic formulas of modelled pressure $P_m(r, \varphi)$ from Wang et al.⁴⁷ and PhyE2E. **c**, Average MSE (left) and complexity (right) when utilizing data from different radii for the models to be compared. **d**, Instrumental observations and formula predictions for plasma-sheet pressure using different models. **e**, Predictions by PhyE2E for plasma-sheet pressure using data from

different radii. **f**, Solar rotation varies at different latitudes, making magnetic field lines stretched and twisted. **g**, MSE and complexity from different models using different numbers of training data. **h**, Predictions of Snodgrass⁵² and PhyE2E across all latitudes. **i**, Predictions of solar atmospheric, using data from various spectral lines in the photosphere and the chromosphere. **j**, PhyE2E predicts consistent formulas with high robustness across various spectral lines.

polar regions (Fig. 4h). Regarding predictions at high latitudes, Hotta and Kusano⁵⁰ overcame the 'convective conundrum' using the supercomputer Fugaku and successfully reproduced solar-like differential rotation. We selected the simulation data from the north and south polar regions and compared them with the baseline models. The formula derived from PhyE2E performed the best in both polar regions, with correlations of 0.9814 and 0.9740, outperforming all baseline models including the formula proposed by Snodgrass.⁵² In addition, we applied our model to different heights in the solar atmosphere, using data from various spectral lines⁵³ in the photosphere (Si I and Fe I) and the chromosphere (H α) (Fig. 4i). Similar formulas are derived across all the spectral lines with remarkable consistency, suggesting

that the differential rotation speed within the Sun follows a regular and predictable pattern (Fig. 4j).

Performance of contribution function of emission lines

Emission lines in the extreme ultraviolet spectrum of the Sun, such as Fe X lines, are often used to observe the solar corona (Hinode/EIS, Solar Orbiter/EUI) (Fig. 5a). Predicting the contribution functions of these lines helps in plasma diagnostics such as temperatures, densities and magnetic fields, which are essential to understand solar phenomena such as flares and coronal mass ejections. The extreme ultraviolet emission lines can also be formed in other types of astrophysical target, including stellar coronae, galactic nuclei and supernovae.

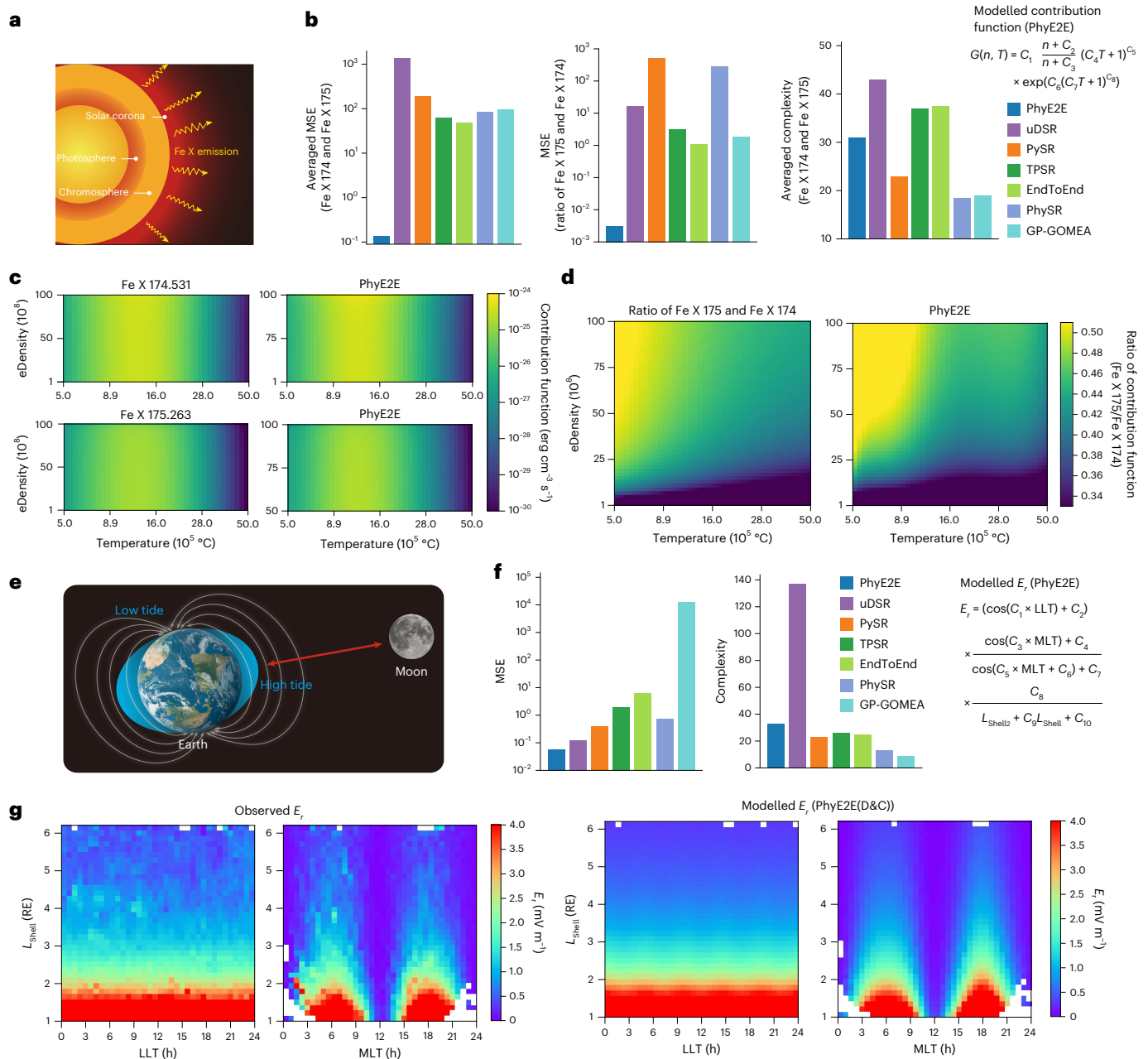


Fig. 5 | Performance of contribution function of emission line predictions and lunar tide signal of plasma layer predictions. a, Emission line in the extreme ultraviolet spectrum of the Sun. **b**, Average MSE for Fe X 174 and Fe X 175 (left), MSE of the ratio between the two emission lines (middle) and complexity (right) of the formulas generated by different models to be compared. **c**, Instrumentally measured contribution function and PhyE2E predictions for Fe X 174 and

Fe X 175. **d**, Instrumentally measured ratio of the two emission lines and PhyE2E predictions. **e**, Tidal radial electric fields influence the Earth's magnetospheric electric fields. **f**, MSE and complexity for different models to be compared. **g**, Instrumentally measured E_r (left) and PhyE2E predictions for dayside and nightside of the Earth.

Understanding the formation of the emission lines can provide insight into the physical conditions and processes of these targets. Given their role as a critical component of fundamental atomic databases applicable to a wide range of studies, considerable efforts have been made to calculate the contribution functions of emission lines (CHIANTI^{54,55} and NIST atomic database⁵⁶). The contribution functions could be approximated by solving complex quantum mechanical equations involving detailed calculations of electron transitions, collisions and radiative transfer, which is usually a computationally expensive process. Therefore, a challenge in physics is whether we can accurately estimate the contribution function using easily observable data, including the temperature and electron density around the Sun. Currently, there is

no physics formula that accurately reveals how the temperature and electron density around the Sun influence the contribution levels of the emitted spectral lines.

To address this problem, we downloaded the Fe X 174 and Fe X 175 line data from the CHIANTI database^{54,55}, and uniformly sampled 2,500 data points in an electron density range of 10^8 – 10^{10} and a temperature range of 5×10^5 – 5×10^6 °C. Data were segmented into low- and high-temperature regions using a cutoff of 2.8×10^6 °C. Instead of fine-tuning or retraining our model, we took the data from the low-temperature region as input to generate the formula and test the prediction performance of the formula in the high-temperature region. In the high-temperature region, PhyE2E achieved a substantially

lower MSE, two orders of magnitude smaller than those of the other baseline models (Fig. 5b left). Both the Fe X 174 and 175 emission lines are highly temperature dependent, with a smaller influence from the electron density. To address this issue, we further investigated the ratio of these two lines, which serves as a powerful diagnostic tool for probing the effects of electron density on the intensity. Compared with the prediction of the two spectral lines individually, the prediction of the ratio of the two lines carries greater physical meaning and is also more challenging. Accurate predictions of the individual spectral lines do not guarantee that their ratio can be predicted accurately (Fig. 5b–d). In this task, our formula achieves an MSE of 3.10×10^{-3} , which is three orders of magnitude lower than the second-best method, EndToEnd (Fig. 5b, middle).

Next, we examined the physical meaning of the formulas generated by different methods. First, the complexity of our formula is not the lowest, but it is the only formula that has the correct physical units among all the baseline methods (Fig. 5b right and Supplementary Tables 17 and 18). In the solar corona, the processes of ionization and recombination, as well as collision and excitation, can be considered decoupled⁵⁷. In our formula, the electron density and temperature also exist in a decoupled form. The dependence of the electron density, following the mathematical form $(n + c_1)/(n + c_2)$, is widely accepted by the space physics community⁵⁸. It captures the behaviour where the intensity increases with electron density at low values but saturates at higher densities because of collisional de-excitation. The temperature term of our formula is composed of the combination of a power-law term and an exponential term. The power-law term dominates at low temperatures, capturing the increase in intensity as more electrons gain sufficient energy to excite the ions; the exponential decay term dominates at high temperatures, reflecting the rapid decrease in intensity due to ionization to higher states. This combination of a power-law term and an exponential term was also adopted by Raymond and Smith⁵⁹. The constants of our formula are given in Supplementary Table 5. For the formulas generated by other methods, some have excessive complexity, making them difficult to interpret, while others have incorrect physical units or are overly simplistic. For instance, the formula produced by PhySO is simple, but does not include the electron density term (Supplementary Table 17).

Performance of lunar-tide signal of plasma layer

The Earth's magnetospheric electric fields, including corotation and convection electric fields, are crucial for understanding the behaviour of charged particles and maintaining the stability of the magnetosphere (Fig. 5e). These fields are responsible for the movement and energization of charged particles, which in turn affect space weather and the interaction between the solar wind and Earth's magnetic field. Before 2023, it was commonly accepted in the scientific community that the electric fields at a specific near-Earth location were solely influenced by the Earth's position relative to the Sun and the distance from the Earth's centre. A recent work indicated that, due to the effects of lunar tidal forces, the electric fields were also related to the Earth's position relative to the Moon⁶⁰. However, due to the complexity of the problem, physicists have been unable to provide a physical formula that links these three important factors: electric fields with the distance from the Earth's centre, the relative positions of the Earth and the Moon and the Earth's position relative to the Sun, which can be represented as lunar local time (LLT), magnetic local time (MLT) and L -shell, respectively.

To address this problem, we collected ~20,000,000 data measured by the Van Allen Probes satellites for L -values of 3–6 from January 2013 to May 2019 from the Radiation Belt Storm Probe Electric Field and Waves Instrument official website⁶¹, and used PhyE2E to generate a formula to predict radial electric field, denoted as E_r , from LLT, MLT and L -shell values. Due to the large volume and high redundancy of data, we divided the entire three-dimensional space near the Earth into $50 \times 50 \times 50$ grids, and then calculated the average value of the

radial electric field within each grid. We randomly sampled 80% of the grids as input for our model and also used them as training data for other baseline models. The remaining data were adopted as test data to evaluate the performance of all models. On the basis of the adaptive threshold for the decomposition of the formula, we found that there are no coupling relationships among these three variables. Therefore, we decomposed the original symbolic regression problem into three subproblems, each containing only one variable. Then, we predicted each univariate function using the end-to-end model (Methods). Without fine-tuning or retraining of the model, the formula generated by our model has an MSE lower than the second-best method by 53.37%, with complexity reduced by 75.91% (Fig. 5f). We examined the effects of LLT and MLT on the radial electric field separately and found that our formula provides a good approximation for the original data. The prediction of our formula is much smoother than the measured data, due to the data smoothing applied within each grid (Fig. 5g). In contrast to the formulas generated by other methods, the formula produced by our model captures multiple physical principles, making it more physically meaningful. First, among all the models, only our model provides an asymmetric prediction between the dayside and nightside of the Earth, suggesting that E_r on the dayside decays more rapidly in the radial direction (L -shell), while on the nightside E_r decays faster in the non-radial direction (MLT). Second, since the radial electric field (positive direction towards Earth) is derived from the calculation of the electric field's y component, it exhibits periodic variation with MLT, and the period is 12 h (ref. 62), which is consistent with the periodicity of the cosine function of MLT in our formula (12.31 h). Third, our formula indicates that E_r decays with the square of the L -shell, which is consistent with theoretical calculations. According to ideal magnetohydrodynamics, the corotational electric field E could be derived as $E = -\Omega_E B_0 / L_{\text{shell}}^2$, where Ω_E and B_0 are Earth's rotational angular velocity and Earth's surface magnetic field, respectively. The constants of our model can be found in Supplementary Table 6. The complexity of our formula is not the lowest among all the models because the pattern of this physical application is complicated (Fig. 5f and Supplementary Tables 19 and 20). Among methods with lower complexity, only the formulas from BSR and PySR exhibit periodicity in LLT or MLT, and only PySR captures the inverse relationship between E_r and L -shell. However, the BSR formula does not include the crucial L -shell variable, and the PySR formula lacks the LLT variable (Supplementary Table 19).

Discussion

Existing symbolic regression research primarily employs search methods based on MCTS and reinforcement learning. These methods often struggle to accurately predict formulas with a large number of variables or complex operational relationships between variables. To discover the correct formulas within a limited time, most of the MCTS approaches require prior knowledge to achieve an initialization close to the true solution. In contrast, our method can decompose formulas without knowing their specific forms, substantially reducing the complexity of symbolic regression. For each decomposed subproblem, we utilized an end-to-end approach, tokenizing the data and directly translating them into formula strings using a transformer. Of all the symbolic regression methods, our approach offers a ready-to-use model and is the only one that does not require retraining or fine-tuning on physical data.

One characteristic of space physics is that there is no data noise on the planet scales or on the microscopic particle scale. However, if the model is to be generalized to other areas of physics, such as condensed matter physics and fluid dynamics, the effects of noise inherent in the data must be considered. Since the data are free from noise, the model must learn to deduce the operational relationships between physical variables based solely on the data provided. Therefore, we can still leverage LLMs using this method of simulating the generation of physics formulas for data augmentation. This principle is similar to that of AlphaZero⁶³, which does not require human game records but can learn

to play Go through AI-versus-AI games. This is because AI only needs to learn optimal strategies in various complex situations, rather than necessarily mimicking human players' thought processes and habits. Currently, our model cannot handle operations such as integration and differentiation, which means that a substantial portion of physics formulas based on partial differential equations cannot be resolved. We believe that the data augmentation and formula decomposition techniques are still applicable in partial differential equations.

Methods

We first detail the components of our system, starting with the generative model for synthetic physics formulas. Next we present the core framework, which includes an end-to-end symbolic regression model integrated with a D&C strategy for decomposing complex formulas into simpler subformulas. We then describe the process to refine the formula predicted by the end-to-end model, encompassing MCTS and genetic programming refinement. Finally we discuss the details of how to construct test data and evaluation metrics.

Generative model for synthetic physics formulas

To generate synthetic formulas resembling real physics formulas, we fine-tuned the pretrained OpenLLaMA2-3B language model⁶⁴ using the AI Feynman dataset²⁷, which is a benchmark collection of mathematical formulas representing real-world physical laws and relationships, consisting of 100 formulas. A two-stage fine-tuning strategy was devised to address the challenge of limited training data and to integrate prior knowledge of physical unit systems into the training process. In the first stage, the OpenLLaMA2-3B model was fine-tuned on the AI Feynman training set. The fine-tuned model was then employed to generate 50,000 synthetic formulas. These formulas were evaluated for consistency with physical unit systems, resulting in approximately 8,000 formulas that adhered to unit consistency. The second stage involved reassigning weights to the 8,000 unit-consistent formulas generated in the first stage. This weighting was designed to ensure that the statistical distribution of the synthetic formulas, such as the number of variables, formula depth and operator frequency, aligned with those of the real physics formulas in the AI Feynman dataset. Finally the language model was further fine-tuned using the weighted distribution of the 8,000 formulas.

Specifically, the fine-tuning was performed using the DeepSpeed ZeRO Stage 2 optimizer within the Hugging Face transformer framework⁶⁵, with a learning rate of 3×10^{-5} . The training prompt followed the format 'Generate a physics formula: {formula}'. Mathematical formulas in the prompt were represented in plain text using their natural mathematical forms. The notation of variables for physical quantities in the formulas adheres to the standard conventions used in the Feynman dataset. To generate synthetic formulas, the model was prompted with the same instruction. The hyperparameters of the generative model were configured to balance diversity and quality in the generated formulas: the temperature was set to 2.0, efficient sampling was enabled (do_sample=True) and the maximum length of the generated sequences was restricted to 64 tokens. To evaluate the consistency of synthetic formulas with physical unit systems (in the first stage), the formula's expression tree⁶⁶ was constructed, and the units of each subtree were computed in a bottom-up manner, starting from the leaves and moving toward the root. During this process, the following checks were performed: (1) for any subtree in the form of $A + B$ or $A - B$, the units of A and B were required to be the same, (2) for any subtree in the form of $\sin(A)$, $\cos(A)$ or $\exp(A)$, the unit of A was required to be null. For instance, the formula 'acceleration + velocity/time' is valid while the formula 'acceleration + velocity' or 'sin(acceleration + velocity/time)' is invalid due to unit mismatch. No constraints were imposed on operations such as multiplication, division or square root, although these operations produce derived units that may affect the validity of their parent expressions. For example, 'sqrt(acceleration/time)' yields

a unit equivalent to that of velocity, so the formula 'sqrt(acceleration/time) + velocity' satisfies the unit consistency requirement for addition. In contrast, 'sqrt(acceleration/time) + velocity/time' fails this criterion. To reassign weights to a set of formulas (in the second stage), a linear program (LP) was formulated. The LP variables represent the weights assigned to each formula, subject to the constraints that the weights must be non-negative and sum up to 1. The objective was to minimize the total variation distances between the statistical distributions (for example, number of variables, formula depth, operator frequency) of the weighted synthetic formulas and those of the AI Feynman dataset. These total variation distances were expressed as linear combinations of the LP variables. Additionally, a regularization term was included in the LP objective to ensure that the weighted distribution did not deviate excessively from the original uniform distribution.

The D&C strategy

Many physics formulas exhibit intrinsic simplicity and symmetry, with variables often interconnected through straightforward addition or multiplication. Inspired by this observation, we proposed a D&C strategy to decompose the target formula into a summation (or multiplication) of simpler subformulas by estimating intervariable relationships. To achieve this, we first train an oracle neural network to fit the data, then use the Hessian matrix to identify the inner non-linear relationship between variables. These relationships guide the decomposition strategy, breaking the target formula into several simpler subformulas, which are then predicted independently and subsequently combined to reconstruct the target formula. For instance, consider the mathematical formula $f(x_1, x_2, x_3, x_4) = x_1 x_2 + x_3 \log(x_4)$. It is straightforward to verify that the second derivatives between the group $\{x_1, x_2\}$ and the group $\{x_3, x_4\}$ are zero, that is, $\partial^2 f / (\partial x_i \partial x_j) = 0$, $\forall i \in \{1, 2\}, \forall j \in \{3, 4\}$, which mathematically indicates that the target formula can be decomposed into two subformulas $f_1(x_1, x_2) = x_1 x_2$ and $f_2(x_3, x_4) = x_3 \log(x_4)$. Our D&C strategy is based on a generalization of the underlying mathematical principle of the above example. Below, we first introduce the inner-variable relationships, followed by the decomposition strategy for subformulas and the resampling technique for data points. Finally, we present the aggregation theorem for the back-aggregation step.

The oracle neural network and estimation of intervariable relationships

Given the data $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ with N evaluations of the target formula $f(\mathbf{x})$ ($N = 200$ in our experiments), an oracle neural network $\tilde{f}_\theta(\mathbf{x})$ was first trained to approximate $f(\mathbf{x})$ at any input point $\mathbf{x}$. Here, θ are the parameters of the oracle neural network, which consists of five hidden layers. The first three layers each contained 128 tanh neurons, while the last two layers each contained 64 tanh neurons. The network was trained for 400 epochs, with an initial learning rate of 0.1 that decayed tenfold every 100 epochs. The following definition characterizes the intervariable relationship to be estimated for decomposing the target formula.

Definition 1. Let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be a univariate operator. Two features $i, j \in \{1, \dots, n\}$ of a target formula $f : \mathbb{R}^n \rightarrow \mathbb{R}$ are said to be σ -separable if there exist subformulas f_1 and f_2 such that f can be expressed as

$$f(\mathbf{x}) = \sigma(f_1(\mathbf{x}_{-i}) + f_2(\mathbf{x}_{-j})),$$

where $\mathbf{x}_{-i}$ is the $(n - 1)$ -dimensional vector obtained by removing $\mathbf{x}_i$ from $\mathbf{x}$, and $\mathbf{x}_{-j}$ is defined analogously.

The univariate operator σ cannot be generalized to a multivariate operator, as our method relies on the invertibility of σ . When the operator σ is invertible, the following lemma, whose proof is provided in Supplementary Note 6.1, provides an equivalent condition to check whether two features are σ -separable in a twice-differentiable formula.

Lemma 1. Let the univariate operator $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ and the target formula $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be twice differentiable. Suppose σ is strictly monotonic, then two features $i, j \in \{1, \dots, n\}$ are σ -separable if and only if, for all $\mathbf{x} \in \mathbb{R}^n$,

$$\frac{\partial^2 \sigma^{-1} \circ f}{\partial x_i \partial x_j}(\mathbf{x}) = (\sigma^{-1})''(f(\mathbf{x})) \cdot \frac{\partial f}{\partial x_i}(\mathbf{x}) \cdot \frac{\partial f}{\partial x_j}(\mathbf{x}) + (\sigma^{-1})'(f(\mathbf{x})) \cdot \frac{\partial^2 f}{\partial x_i \partial x_j}(\mathbf{x}) = 0. \quad (1)$$

Practical implementation

In our experiment, we tried the univariate operators $\sigma \in \{\text{id}, \text{sqrt}, \text{inv}, \text{arccos}, \text{arcsin}, \log, \text{sqrt} \circ \log, \text{inv} \circ \log, \text{arcsin} \circ \log, \text{arccos} \circ \log\}$ to perform the D&C strategy and predict the target formula via the end-to-end model (Methods). Note that the operators that involve the logarithm effectively separate the target formula into the multiplication of two subformulas according to Definition 1. The predictions on the basis of different univariate operators were collected and fed into the MCTS and genetic programming module for further refinement (Methods).

However, even with access to the oracle neural network $\tilde{f}_\theta$, we cannot directly verify the condition in Lemma 1 because it requires evaluation of all the points in $\mathbf{x} \in \mathbb{R}^n$, and the second-order derivative of the approximate function $\tilde{f}_\theta(\mathbf{x})$ tends to be noisy. In our algorithm, this condition was verified approximately by sampling a subset of points and employing a majority rule to mitigate the effects of approximation noise. Specifically, for the set of training data points $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$ ($N = 200$), two features i and j are determined to be σ -separable if, for a threshold parameter $\varepsilon > 0$, it holds that

$$J_{i,j}(\sigma, \tilde{f}_\theta) \stackrel{\text{def}}{=} \text{median}_{1 \leq k \leq N} \left\{ \left| \frac{\partial^2 \sigma^{-1} \circ \tilde{f}_\theta}{\partial x_i \partial x_j}(\mathbf{x}_k) \right| \right\} \leq \varepsilon. \quad (2)$$

The choice of ε is crucial to the accurate identification of σ -separable pairs. We first employ the ‘fixed thresholding’ strategy, treating $\varepsilon = \varepsilon(\sigma)$ as a constant for each univariate operator σ . This fixed constant is determined using a data-driven approach for each σ . More specifically, we randomly sampled 1,000 target formulas h_k and trained the corresponding oracle neural networks $\tilde{h}_k(\mathbf{x}_k; \theta_k)$. For each feature pair $\{i, j\}$, we identified their σ -separability by Lemma 1 and calculated the corresponding $J_{i,j}(\sigma, \tilde{h}_k)$, where J is the median derivative defined by equation (2). The threshold was then chosen to maximize the number of feature pairs among the 1,000 additional target formulas that were classified correctly in terms of σ -separability.

On the other hand, a single constant value for a fixed ε may not work well for all target formulas f , as different functions can exhibit vastly different scales of derivatives. This scale can even vary if f is multiplied by a large constant factor. To address this issue, the technique of ‘adaptive thresholding’ was also proposed to automatically select the threshold on the basis of the derivative scale, thereby improving the numerical stability of the estimation of σ -separable pairs. More specifically, let $\varepsilon_0 = \min_{1 \leq i < j \leq n} J_{i,j}(\sigma, \tilde{f}_\theta)$. We then define $\varepsilon_1 = \alpha_{\min} \varepsilon_0$ and $\varepsilon_2 = \alpha_{\max} \varepsilon_0$ as the minimum and maximum thresholds, respectively. In our algorithm, we set $\alpha_{\min} = 2$ and $\alpha_{\max} = 10$. For each $\varepsilon \in [\alpha_{\min}, \alpha_{\max}]$, the set of σ -separable pairs (hereafter referred to as the σ -separable set) was estimated as

$$Q_\varepsilon(\sigma, \tilde{f}_\theta) \stackrel{\text{def}}{=} \{(i, j) \mid 1 \leq i < j \leq n, J_{i,j}(\sigma, \tilde{f}_\theta) \leq \varepsilon\}. \quad (3)$$

Finally, the class of σ -separable sets was defined as

$$\mathcal{Q} \stackrel{\text{def}}{=} \{Q(\sigma, \tilde{f}_\theta) \mid \varepsilon \in [\alpha_{\min}, \alpha_{\max}]\}. \quad (4)$$

It is straightforward to verify that $|\mathcal{Q}| \leq n(n-1)/2$. Therefore, applying the D&C strategy on the basis of each $Q \in \mathcal{Q}$ is computationally feasible. Each application of this strategy derived a prediction of the

target formula. These formulas were evaluated and the best one was selected. In the following subsections, we will explain how the D&C strategy operates for any estimated σ -separable set Q : it begins by inducing a set of feature sets, followed by predicting the subformulas for these feature sets and ultimately integrating them into the final prediction of the target formula.

The division of the target formula

We have discussed the σ -separable relationship between pairs of features. The following definition of the global division of the target formula and features is crucial to our D&C strategy.

Definition 2. Let $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be a univariate operator. For any feature subset A_i of $A = \{1, 2, \dots, n\}$, denote by $\mathbf{x}_{A_i}$ the vector obtained by restricting $\mathbf{x}$ to A_i , that is, if $A_i = \{j_1, j_2, \dots, j_k\}$ then $\mathbf{x}_{A_i} = \{x_{j_1}, x_{j_2}, \dots, x_{j_k}\}$. A class of subsets $\{A_i\}_{i=1}^m$ is said to be a σ -division of f if none of the subsets is contained in another subset and there exist m subformulas $f_1, f_2, \dots, f_m$ such that f can be expressed as

$$f(\mathbf{x}) = \sigma(f_1(\mathbf{x}_{A_1}) + f_2(\mathbf{x}_{A_2}) + \dots + f_m(\mathbf{x}_{A_m})), \quad (5)$$

where each f_i is a function of $\mathbf{x}_{A_i}$.

It is quite straightforward to derive a σ -division from σ -separable pairs, detailed as follows. First, the σ -separable relationships between all features are identified as described above. Then, the algorithm starts with an initial (and trivial) σ -division $\mathcal{A} = \{\{1, 2, \dots, n\}\}$ and iteratively refines it. At each iteration, the algorithm selects a σ -separable pair of features j_1 and j_2 that has not been considered. For each feature subset $A \in \mathcal{A}$ such that $\{j_1, j_2\} \subseteq A$, $\mathcal{A}$ is updated as follows:

$$\mathcal{A} \leftarrow (\mathcal{A} - \{A\}) \cup \{A - \{j_1\}, A - \{j_2\}\}.$$

This process is done after all σ -separable feature pairs have been considered. By Lemma 2 below, whose proof can be found in Supplementary Note 6.1, this iterative procedure is guaranteed to yield a valid σ -division.

Lemma 2. Let univariate operator $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ be strictly monotonic and $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be the target formula that is twice differentiable. Suppose we have accurately obtained the set of all σ -separable feature pairs of f . Then, for each iteration number $\ell \geq 1$, the division obtained by the above algorithm after the ℓ th iteration, denoted by $\{A_k^\ell\}_{k=1}^{m_\ell}$, is a σ -division of f .

Algorithm 1. Algorithm 1 is provided in Supplementary Note 5 to formally describe the above procedure to derive σ -separable feature pairs via the fixed and adaptive thresholding techniques, as well as to construct a σ -division based on the σ -separable pairs.

Evaluation of surrogate subformulas and back-aggregation for the target formula

Once a σ -division $\{A_i\}_{i=1}^m$ is derived, it would be most natural to recursively perform symbolic regression to predict the subformulas f_i and reconstruct the target formula f according to equation (5). However, there are two challenges to this approach. First, we do not have evaluation data $\{(\mathbf{x}, y = f(\mathbf{x}))\}$ for each subformula f_i . Moreover, the subformulas f_i are not even unique. For example, if $f(x_1, x_2, x_3) = \sigma(f_1(x_1, x_2) + f_2(x_1, x_3))$, then it can also be expressed as $f(x_1, x_2, x_3) = \sigma((f_1(x_1, x_2) - x_1) + (f_2(x_1, x_3) + x_1))$, where $f'_1 = f_1 - x_1$ and $f'_2 = f_2 + x_1$ are also a set of valid subformulas for the σ -division $\{\{x_1, x_2\}, \{x_1, x_3\}\}$.

To address the above challenges, we turn to predicting the surrogate subformulas $\{g_i\}_{i=1}^m$, defined as follows. First, an arbitrary $\mathbf{z} \in \mathbb{R}^n$ is chosen, where in the experiment $\mathbf{z}$ was sampled from the standard multivariate Gaussian distribution. Then, given the σ -division $\{A_i\}_{i=1}^m$, for each $i \in \{1, 2, \dots, m\}$, we define $g_i : \mathbb{R}^{A_i} \rightarrow \mathbb{R}$ as

$$g_i(\mathbf{x}_{A_i}) = f_i(\mathbf{x}_{A_i}, \mathbf{x}_{\bar{A}_i} = \mathbf{z}_{\bar{A}_i}), \quad (6)$$

where $\bar{A}_i = [n] - A_i$.

Each surrogate subformula g_i might be quite different from the corresponding subformula f_i . For instance, consider $f(x_1, x_2, x_3) = f_1(x_1, x_3) + f_2(x_2, x_3)$ where $f_1(x_1, x_3) = \sin(x_1 x_3)$ and $f_2(x_2, x_3) = \cos(x_2 x_3)$. According to the definition, we have $g_1(x_1, x_3) = \sin(x_1 x_3) + \cos(z_2 x_3)$ and $g_2(x_2, x_3) = \cos(x_2 x_3) + \sin(z_1 x_3)$. Note that each g_i not only contains f_i , but also introduces extra non-trivial terms. The following theorem, the proof of which can be found in Supplementary Note 6.2, provides a way to eliminate the extra terms and aggregate the surrogate subformulas $\{g_i\}_{i=1}^m$ to reconstruct the target formula f .

Theorem 3. Suppose the univariate operator $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is strictly monotonic. Let $\mathcal{A} = \{\mathcal{A}_i\}_{i=1}^m$ be a σ -division of the target formula f and $\{g_i\}_{i=1}^m$ be defined as in equation (6) based on a vector $\mathbf{z}$. For each $I \subseteq [m]$, denote

$$\mathcal{A}_I = \bigcap_{i \in I} \mathcal{A}_i.$$

Then, it holds that

$$f(\mathbf{x}) = \sigma \left(\sum_{\emptyset \neq I \subseteq [m]} \frac{(-1)^{|I|-1}}{|I|} \sum_{i \in I} \sigma^{-1} \circ g_i(\mathbf{x}_{\mathcal{A}_I}, \mathbf{x}_{\bar{\mathcal{A}}_I} = \mathbf{z}_{\bar{\mathcal{A}}_I}) \right). \quad (7)$$

Practical implementation

For the i th surrogate subformula g_i , the data $D^{(i)} = \{\mathbf{x}_k^{(i)}, y_k^{(i)}\}_{k=1}^N$ were constructed as follows:

$$\mathbf{x}_k^{(i)} = (\mathbf{x}_k)_{A_i} y_k^{(i)} = \tilde{f}_\theta(\mathbf{x}_{A_i} = \mathbf{x}_k^{(i)}, \mathbf{x}_{\bar{A}_i} = \mathbf{z}_{\bar{A}_i})$$

where $\mathbf{x}_k$ is the k th data point in the original data D . Then, each g_i was predicted using the end-to-end model (as will be described below). Finally, the target formula f was predicted by aggregating the surrogate subformulas according to equation (7).

Architecture of end-to-end model

Kamienny et al.²⁴ established a transformer-based end-to-end model for symbolic regression. We design a similar transformer that includes 4 encoder layers and 16 decoder layers, forming an asymmetric architecture⁶⁷. Each layer consists of 16 attention heads and an embedding dimension of 512. We remove positional embeddings to ensure permutation invariance of the input data.

Physical priors (for example physical units) play a crucial role in governing the structure and plausibility of physics formulas^{33,68}. In PhyE2E, we integrate four types of physical prior, including the physical units of input and output variables, formula complexity, candidate operators and candidate constants. Besides the evaluations at N data points, we also append a set of h candidate physical priors to the input of our model.

To encode the observed data points, symbolic formulas and physical priors, we construct a vocabulary that includes tokens for float numbers, operators, variables and physical units. Each float number is decomposed into three tokens representing its sign, mantissa (between 0 and 9,999) and exponent (from 10^{-100} to 10^{+100})⁶⁷. For physical units, we adopt five commonly used base units: metre (m), second (s), kilogram (kg), Kelvin (K) and volt (V)²⁷, which are slightly different from the International System of Units. In this way, a physical unit can be encoded into five tokens. For dimensionless constants and formulas, we simply assign each base unit with value 0. Each data point is encoded into $(n+1) \times 3$ tokens, where each of the n features and the evaluation is encoded into three tokens. Each of the h physical priors is represented using specialized vocabulary tokens that encode the variables, physical units, operators and float numbers. Specifically,

we use 1 + 5 tokens to represent a variable with its physical unit, and 3 + 5 tokens to represent a constant with its physical unit. All data point and physical prior sequences are padded with the <pad> token to ensure a uniform fixed length $L = 33$. They are finally concatenated together to form the input of our model.

Additionally, to help the model better understand a consistent physical unit system, we propose a novel unit decoding strategy for the model's output. Specifically, we incorporate the physical units of each operator and variable within the formula into the output sequence. While formulas are represented as prefix expressions composed of operators and variables, we enhance this representation by associating each operator and variable with its corresponding physical unit, so that our model outputs a sequence of (operator/variable, physical unit) pairs. Notably, we do not impose explicit unit consistency constraints during generation. Instead, by explicitly inferring the physical units of the formula in a step-by-step manner within the output sequence, the model is guided to learn the underlying physical principles that govern variable relationships automatically. This approach enables consistent and meaningful unit inference directly from data.

Training details

The training data consisted of the data evaluated using -200,000 synthetic physics formulas generated above. Each formula was evaluated at $N = 200$ points sampled from the standard multivariate Gaussian distribution, conditioned on the fact that the formula is properly defined at the point, which follows the previous work²⁴.

We used the cross-entropy loss on the next-token prediction with the Adam optimizer, starting with a learning rate of 10^{-7} , gradually increased to 2×10^{-4} during the first 10,000 steps, and subsequently decaying proportional to the inverse square root of the step count⁶⁹. A validation set comprising 20,000 synthetic formulas is held out, and our model is trained for 100 epochs, processing 500 million formulas in total, until the validation accuracy stabilizes. Each batch consists of 10,000 tokens, with formulas grouped by similar lengths to minimize padding overhead. The training was performed on a single NVIDIA A100 graphics processing unit with 80 GB of memory over about 1 d.

Constant optimization

The end-to-end model only returned a 'formula skeleton', where the constant parameters in the formula remained unoptimized. We adopted the Broyden–Fletcher–Goldfarb–Shanno algorithm⁷⁰ to further refine the constants in the formula skeleton. Specifically, given a formula f , we denote by $\mathbf{c}$ the constant parameters in the formula. On the basis of the data $\{\mathbf{x}_k, y_k\}_{k=1}^N$, the BFGS algorithm was invoked to find

$$\underset{\mathbf{c}}{\operatorname{argmin}} \sum_{k=1}^N [f(\mathbf{x}_k; \mathbf{c}) - y_k]^2.$$

MCTS refinement

The standard MCTS process consists of four steps²⁰: (1) selection, where the best candidate formulas are identified on the basis of their performance; (2) expansion, where new symbolic formulas (which may be incomplete) are generated by adding one more operator from the incomplete formula; (3) simulation, where the incomplete formulas are randomly simulated and then evaluated on the basis of their predictive accuracy; (4) back-propagation, where the results are propagated back to update rewards and visit times of each MCTS node. We employed MCTS to refine the target formula obtained using the end-to-end model. Specifically, random subtrees were removed from the expression tree of the target formula, and MCTS was invoked with the resulting incomplete formula to search for a better target formula. Moreover, following the work of Sun et al.²⁰, we constructed a grammar pool to reduce the search space of MCTS. Specifically, this grammar pool consisted of basic operators (+, −, ×, /, sin(x), cos(x),

$\exp(x)$) and the operators extracted from the predicted results of the end-to-end model. During the expansion steps, MCTS was restricted to select operators only from the grammar pool to construct a complete symbolic formula.

Genetic programming

Our genetic programming refinement module followed the work of PySR²⁸, where we used the results from the end-to-end model and MCTS as the initial populations for the genetic programming algorithm. During each round of evolution, each formula in the population undergoes random mutations, including appending, changing or deleting specific operators in the formula. Crossover operations were also performed between individuals to combine their expressions and create new formulas. The entire genetic programming algorithm optimizes the formulas for 40 rounds. The optimization process stops early if the MSE of one of the generated formulas reaches 10^{-6} during the process. Finally, a Pareto front of the formulas was generated. The best formula from this set was selected on the basis of the criterion $\text{MSE} \times 0.99^{\text{complexity}}$ to serve as the final solution, considering a trade-off between accuracy and complexity.

Details of test data

To ensure that no data leakage occurred, we strictly split the training and test datasets. While it is not straightforward to identify mathematically equivalent formulas via symbolic derivations, we chose to measure the similarity between two formulas on the basis of numerical evaluations. Specifically, we define the similarity between two formulas, f_i and f_j , with the same number of features, using the averaged R^2 score:

$$\text{sim}(f_i, f_j) \stackrel{\text{def}}{=} 1 - \frac{1}{2} \left[\frac{\sum_{k=1}^{N'} (f_i(\mathbf{x}_k) - f_j(\mathbf{x}_k))^2}{\sum_{k=1}^{N'} (f_i(\mathbf{x}_k) - \bar{f}_i)^2} + \frac{\sum_{k=1}^{N'} (f_j(\mathbf{x}_k) - \bar{f}_j)^2}{\sum_{k=1}^{N'} (f_j(\mathbf{x}_k) - \bar{f}_j)^2} \right],$$

where $\bar{f}_i \stackrel{\text{def}}{=} [\sum_{k=1}^{N'} f_i(\mathbf{x}_k)]/N'$, $N' = 40$ and the $\mathbf{x}_k$ are independently sampled from the standard multivariate Gaussian distribution. The similarity is set to 0 if there exists an $\mathbf{x}_k$ such that exactly one of $f_i(\mathbf{x}_k)$ and $f_j(\mathbf{x}_k)$ is undefined. The similarity between two formulas with different numbers of features is also set to 0. Observe that two mathematically equivalent formulas achieve the maximum possible similarity of 1.

We selected 3,000 formulas such that the pairwise similarity is less than 0.99 to form our test set. For the training dataset, we only removed the symbolically identical formulas while not requiring the pairwise similarity to be away from 1. This is because formulas that are mathematically equivalent but with different symbolic forms may help the model understand the mathematical equivalences. Moreover, such formulas might originate from different physical scenarios and have different physical units, thus still represent distinct entities. The formulas in the test set were further divided into three difficulty levels on the basis of their maximum similarity with the formulas in the training dataset. Formulas with a similarity greater than 0.95 were defined as the ones that were easy to predict, those with a similarity between 0.8 and 0.95 were considered of medium difficulty and formulas with a similarity less than 0.8 were considered hard.

Evaluation metrics

We evaluate the performance of our model and other baseline models using the following metrics.

- **Symbolic accuracy.** This evaluation metric was introduced by La Cava et al.³⁹. Let the target formula be f . The symbolic accuracy of a predicted formula g is 1 if there exists a constant such that $f - g \equiv c$ or $f = c \cdot g$ ($c \neq 0$ in the second case); otherwise, the symbolic accuracy is 0.
- **Numerical precision.** This metric is used to evaluate the numerical precision of data points given by the predicted formula. We utilize the R^2 -score for the predicted formula g on

testing data points $\{\mathbf{x}_k^{\text{test}}, y_k^{\text{test}}\}_{k=1}^N$ ($N = 200$) which are sampled from the same distribution as the training data points:

$$R^2 = 1 - \frac{\sum_{k=1}^N (y_k^{\text{test}} - g(\mathbf{x}_k^{\text{test}}))^2}{\sum_{k=1}^N (y_k^{\text{test}} - \bar{y})^2}$$

where $\bar{y} = \frac{1}{N} \sum_{k=1}^N y_k^{\text{test}}$.

- **Accuracy of physical units.** This metric aims to assess the capability of a model to generate formulas with consistent physical units. The accuracy is 1 if all operations in the formula work with compatible physical units and the physical unit of the result is the same as the target formula; otherwise, the accuracy of physical units is 0.
- **Formula complexity.** The complexity of a formula f is defined to be the number of operators, variables and constant parameters in the formula. We also define the relative complexity of a prediction g as the difference (in absolute value) between the complexity of g and that of the target formula f .
- **Formula depth.** The depth of a formula f is the maximum depth of its expression tree.

Data availability

AI Feynman data can be downloaded from the Feynman Symbolic Regression Database (<https://space.mit.edu/home/tegmark/aifeynman.html>). The formula generated by the LLM, including the training and test datasets, can be downloaded⁷¹ from https://figshare.com/articles/dataset/PhyE2E_dats/29615831/1. The SSN data can be downloaded from the SILSO website (<https://www.sidc.be/SILSO/datafiles>). The plasma pressure data can be downloaded from the Geotail and Time History of Events and Macroscale Interactions during Substorms website (https://themis.igpp.ucla.edu/overview_data.shtml). The solar rotational angular velocity data can be found in the table presented by Snodgrass in 'Magnetic rotation of the solar photosphere'⁵². We collected the contribution function of emission lines data from the CHIANTI website (<http://www.chiantidatabase.org>). Lunar tide signal data were downloaded from the Radiation Belt Storm Probe Electric Field and Waves Instrument official website (<https://www.space.umn.edu/rbspefw-data/>). Source data are provided with this paper.

Code availability

Codes for running PhyE2E including both training and test modules are accessible at <https://github.com/Jie0618/PhysicsRegression> with a permanent version available⁷² via Zenodo at <https://doi.org/10.5281/zenodo.16305086>. The pretrained PhyE2E model can be downloaded at https://figshare.com/articles/dataset/PhyE2E_dats/29615831/1.

References

1. Cranmer, M. D. *Interpretable Machine Learning for the Physical Sciences*. PhD thesis, Princeton Univ. (2023).
2. Pearce Williams, L. Faraday's discovery of electromagnetic induction. *Contemp. Phys.* **5**, 28–37 (1963).
3. Brunton, S. L., Proctor, J. L. & Kutz, J. N. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proc. Natl Acad. Sci. USA* **113**, 3932–3937 (2016).
4. De Florio, M., Kevrekidis, I. G. & Karniadakis, G. E. AI-Lorenz: a physics-data-driven framework for black-box and gray-box identification of chaotic systems with symbolic regression. *Chaos Solitons Fractals* **188**, 115538 (2024).
5. Ahmadi Daryakenari, N., De Florio, M., Shukla, K. & Karniadakis, G. E. AI-Aristotle: a physics-informed framework for systems biology gray-box identification. *PLoS Comput. Biol.* **20**, e1011916 (2024).

6. Schmidt, M. D. & Lipson, H. Age-fitness Pareto optimization. In *Proc. 12th Annual Conference on Genetic and Evolutionary Computation* 543–544 (Association for Computing Machinery, 2010).
7. Schmidt, M. & Lipson, H. Distilling free-form natural laws from experimental data. *Science* **324**, 81–85 (2009).
8. La Cava, W., Helmuth, T., Spector, L. & Moore, J. H. A probabilistic and multi-objective analysis of lexibase selection and ϵ -lexibase selection. *Evol. Comput.* **27**, 377–402 (2019).
9. La Cava, W., Singh, T. R., Taggart, J., Suri, S. & Moore, J. H. Learning concise representations for regression by evolving networks of trees. In *7th International Conference on Learning Representations* Vol. 6 (ed. Sainath, T.) 3987–4002 (ICLR, 2019).
10. Virgolin, M., Alderliesten, T., Witteveen, C. & Bosman, P. A. Improving model-based genetic programming for symbolic regression of small expressions. *Evol. Comput.* **29**, 211–237 (2021).
11. McCormick, T. gplearn: genetic programming in Python. *GitHub* <https://github.com/trevorstephens/gplearn> (2019).
12. De Franca, F. & Aldeia, G. Interaction-transformation evolutionary algorithm for symbolic regression. *Evol. Comput.* **29**, 367–390 (2021).
13. Arnaldo, I., Krawiec, K. & O'Reilly, U.-M. Multiple regression genetic programming. In *Proc. 2014 Annual Conference on Genetic and Evolutionary Computation* (ed. Igel, C.) 879–886 (Association for Computing Machinery, 2014).
14. Kommenda, M., Burlacu, B., Kronberger, G. & Affenzeller, M. Parameter identification for symbolic regression using nonlinear least squares. *Genet. Program. Evolvable Mach.* **21**, 471–501 (2020).
15. Virgolin, M., Alderliesten, T. & Bosman, P. A. Linear scaling with and within semantic backpropagation-based genetic programming for symbolic regression. In *Proc. Genetic and Evolutionary Computation Conference* (ed. López-Ibáñez, M.) 1084–1092 (Association for Computing Machinery, 2019).
16. Kamienny, P.-A., Lample, G., Lamprier, S. & Virgolin, M. Deep generative symbolic regression with Monte-Carlo-tree-search. *Proc. Mach. Learn. Res.* **202**, 15655–15668 (2023).
17. Lu, Q., Tao, F., Zhou, S. & Wang, Z. Incorporating actor-critic in Monte Carlo tree search for symbolic regression. *Neural Comput. Appl.* **33**, 8495–8511 (2021).
18. Xu, Y., Liu, Y. & Sun, H. Reinforcement symbolic regression machine. In *Proc. 12th International Conference on Learning Representations* (ICLR, 2024).
19. Xie, Y. et al. An efficient and generalizable symbolic regression method for time series analysis. Preprint at <http://arxiv.org/abs/2409.03986> (2024).
20. Sun, F., Liu, Y., Wang, J.-X. & Sun, H. Symbolic physics learner: discovering governing equations via Monte Carlo tree search. In *Proc. 11th International Conference on Learning Representations* (ICLR, 2023).
21. Valipour, M., You, B., Panju, M. & Ghodsi, A. SymbolicGPT: a generative transformer model for symbolic regression. Preprint at <https://arxiv.org/abs/2106.14131> (2021).
22. Chen, T., Li, Z., Xu, P. & Zheng, H. Bootstrapping OTS-Funcim pre-training model (Botfip): a comprehensive multimodal scientific computing framework and its application in symbolic regression task. *Complex Intell. Syst.* **11**, 417 (2025).
23. Xing, H., Salleb-Aouissi, A. & Verma, N. Automated symbolic law discovery: a computer vision approach. In *Proc. AAAI Conference on Artificial Intelligence* 660–668 (AAAI, 2021).
24. Kamienny, P.-A., d'Ascoli, S., Lample, G. & Charton, F. End-to-end symbolic regression with transformers. *Adv. Neural Inf. Process. Syst.* **35**, 10269–10281 (2022).
25. Biggio, L., Bendinelli, T., Neitz, A., Lucchi, A. & Parascandolo, G. Neural symbolic regression that scales. *Proc. Mach. Learn. Res.* **139**, 936–945 (2021).
26. Shojaei, P., Meidani, K., Barati Farimani, A. & Reddy, C. Transformer-based planning for symbolic regression. *Adv. Neural Inf. Process. Syst.* **36**, 45907–45919 (2023).
27. Udrescu, S.-M. & Tegmark, M. AI Feynman: a physics-inspired method for symbolic regression. *Sci. Adv.* **6**, eaay2631 (2020).
28. Cranmer, M. Interpretable machine learning for science with PySR and SymbolicRegression.jl. Preprint at <https://arxiv.org/abs/2305.01582> (2023).
29. Burlacu, B., Kronberger, G. & Kommenda, M. Operon C++: an efficient genetic programming framework for symbolic regression. In *Proc. 2020 Genetic and Evolutionary Computation Conference Companion* 1562–1570 (Association for Computing Machinery, 2020).
30. Grayeli, A., Sehgal, A., Costilla Reyes, O., Cranmer, M. & Chaudhuri, S. Symbolic regression with a learned concept library. *Adv. Neural Inf. Process. Syst.* **37**, 44678–44709 (2024).
31. Shojaei, P., Meidani, K., Gupta, S., Farimani, A. B. & Reddy, C. K. LLM-SR: scientific equation discovery via programming with large language models. In *Proc. 13th International Conference on Learning Representations* (ICLR, 2025).
32. Landajuela, M. et al. A unified framework for deep symbolic regression. *Adv. Neural Inf. Process. Syst.* **35**, 33985–33998 (2022).
33. Tenachi, W., Ibata, R. & Diakogiannis, F. I. Deep symbolic regression for physics guided by units constraints: toward the automated discovery of physical laws. *Astrophys. J.* **959**, 99 (2023).
34. Udrescu, S.-M. et al. AI Feynman 2.0: Pareto-optimal symbolic regression exploiting graph modularity. *Adv. Neural Inf. Process. Syst.* **33**, 4860–4871 (2020).
35. Scholl, P., Bieker, K., Hauger, H. & Kutyniok, G. ParFam—(neural guided) symbolic regression via continuous global optimization. In *Proc. 13th International Conference on Learning Representations* (ICLR, 2025).
36. Liu, Z. et al. KAN: Kolmogorov–Arnold networks. In *Proc. 13th International Conference on Learning Representations* (ICLR, 2025).
37. Liu, Z., Ma, P., Wang, Y., Matusik, W. & Tegmark, M. KAN 2.0: Kolmogorov–Arnold networks meet science. Preprint at <https://arxiv.org/abs/2408.10205> (2024).
38. Jin, Y., Fu, W., Kang, J., Guo, J. & Guo, J. Bayesian symbolic regression. Preprint at <https://arxiv.org/abs/1910.08892> (2019).
39. La Cava, W. et al. Contemporary symbolic regression methods and their relative performance. *Adv. Neural Inf. Process. Syst.* **2021**, 1–16 (2021).
40. Feynman, R., Leighton, R. & Sands, M. *The Feynman Lectures on Physics*, Vol. 1: *The New Millennium Edition: Mainly Mechanics, Radiation, and Heat* (Basic Books, 2011).
41. Feynman, R., Leighton, R. & Sands, M. *The Feynman Lectures on Physics*, Vol. 2 (Pearson/Addison-Wesley, 1963).
42. Feynman, R., Leighton, R. & Sands, M. *The Feynman Lectures on Physics*, Vol. 3 (Pearson/Addison-Wesley, 1963).
43. SILSO World Data Center. The International Sunspot Number (1749–2023). *International Sunspot Number Monthly Bulletin and Online Catalogue* (2023).
44. Hathaway, D. H., Wilson, R. M. & Reichmann, E. J. The shape of the sunspot cycle. *Sol. Phys.* **151**, 177–190 (1994).
45. Upton, L. A. & Hathaway, D. H. Solar cycle precursors and the outlook for cycle 25. *J. Geophys. Res. Space Phys.* **128**, e2023JA031681 (2023).
46. Brehm, N. et al. Eleven-year solar cycles over the last millennium revealed by radiocarbon in tree rings. *Nat. Geosci.* **14**, 10–15 (2021).
47. Wang, C.-P. et al. Empirical modeling of plasma sheet pressure and three-dimensional force-balanced magnetospheric magnetic field structure: 1. Observation. *J. Geophys. Res. Space Phys.* **118**, 6154–6165 (2013).

48. Yue, C., Wang, C.-P., Zaharia, S. G., Xing, X. & Lyons, L. Empirical modeling of plasma sheet pressure and three-dimensional force-balanced magnetospheric magnetic field structure: 2. Modeling. *J. Geophys. Res. Space Phys.* **118**, 6166–6175 (2013).
49. Lui, A. T. & Hamilton, D. C. Radial profiles of quiet time magnetospheric parameters. *J. Geophys. Res. Space Phys.* **97**, 19325–19332 (1992).
50. Hotta, H. & Kusano, K. Solar differential rotation reproduced with high-resolution simulation. *Nat. Astron.* **5**, 1100–1102 (2021).
51. Vasil, G. M. et al. The solar dynamo begins near the surface. *Nature* **629**, 769–772 (2024).
52. Snodgrass, H. B. Magnetic rotation of the solar photosphere. *Astrophys. J.* **270**, 288–299 (1983).
53. Rao, S. et al. Height-dependent differential rotation of the solar atmosphere detected by CHASE. *Nat. Astron.* **8**, 1102–1109 (2024).
54. Dere, K. P., Landi, E., Mason, H. E., Monsignori Fossi, B. C. & Young, P. R. CHIANTI—an atomic database for emission lines. *Astron. Astrophys. Suppl. Ser.* **125**, 149–173 (1997).
55. Dufresne, R. P. et al. CHIANTI—an atomic database for emission lines—paper. XVIII. Version 11, advanced ionization equilibrium models: density and charge transfer effects. *Astrophys. J.* **974**, 71 (2024).
56. Kramida, A., Ralchenko, Y., Reader, J. & NIST ASD Team. *NIST Atomic Spectra Database v.5.12* (NIST, accessed 20 October 2024); <https://physics.nist.gov/asd>
57. Aschwanden, M. J. *Physics of the Solar Corona. An Introduction* (Praxis-Springer, 2004).
58. Mason, H. E. & Monsignori Fossi, B. C. Spectroscopic diagnostics in the VUV for solar and stellar plasmas. *Astron. Astrophys. Rev.* **6**, 123–179 (1994).
59. Raymond, J. C. & Smith, B. W. Soft X-ray spectrum of a hot plasma. *Astrophys. J. Suppl. Ser.* **35**, 419–439 (1977).
60. Xiao, C. et al. Evidence for lunar tide effects in Earth's plasmasphere. *Nat. Phys.* **19**, 486–491 (2023).
61. *RBSP/EFW Data* (Univ. Minnesota, accessed 24 September 2025); <http://www.space.umn.edu/rbsp/efw-data/>
62. Zhang, Z., Liu, W. L., Zhang, D. J. & Cao, J. B. Estimating the corotation lag of the plasmasphere based on the electric field measurements of the Van Allen Probes. *Adv. Space Res.* **73**, 758–766 (2024).
63. Silver, D. et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science* **362**, 1140–1144 (2018).
64. Touvron, H. et al. Llama 2: open foundation and fine-tuned chat models. Preprint at <https://arxiv.org/abs/2307.09288> (2023).
65. Maurya, A., Ye, J., Rafique, M. M., Cappello, F. & Nicolae, B. Deep optimizer states: towards scalable training of transformer models using interleaved offloading. In *Proc. 25th International Middleware Conference*, 404–416 (Association for Computing Machinery, 2024).
66. Lample, G. & Charton, F. Deep learning for symbolic mathematics. In *International Conference on Learning Representations (ICLR, 2020)*.
67. Charton, F. Linear algebra with transformers. *Trans. Mach. Learn. Res.* (2022).
68. Bendinelli, T., Biggio, L. & Kamienny, P.-A. Controllable neural symbolic regression. *Proc. Mach. Learn. Res.* **202**, 2063–2077 (2023).
69. Vaswani, A. et al. Attention is all you need. *Adv. Neural Inf. Process. Syst.* **30**, 5998–6008 (2017).
70. Fletcher, R. *Practical Methods of Optimization* 2nd ed. (Wiley, 1987).
71. Ying, J. *PhyE2E_datas*. *figshare* https://figshare.com/articles/dataset/PhyE2E_datas/29615831 (2025).
72. Ying, J. Jie0618/PhysicsRegression: code for “A neural symbolic model for space physics” version v1.0.0. *Zenodo* <https://doi.org/10.5281/zenodo.16305086> (2025).

Acknowledgements

This work is supported by the National Natural Science Foundation of China grants 52494974 and 62377030, China's Village Science and Technology City Key Technology funding and Wuxi Research Institute of Applied Technologies.

Author contributions

S.-T.Y., Y.Z. and J.M. conceived of the research idea. H.L. and Y.L. were responsible for the construction of datasets. J.Y. was responsible for the training and validation for PhyE2E, the comparison with other baseline methods and the analysis of results. C.Y. and Q.S. conceived of the five applications of space physics. J.Y. implemented the code and conducted the experiments. C.Y. was responsible for the applications of SSN prediction and plasma pressure prediction. Y.C. and Q.S. were responsible for the applications of solar rotational angular velocity prediction and contribution function of emission lines. C.X. was responsible for the application of the lunar tide signal of the plasma layer. J.Y., H.L. and C.Y. wrote the initial draft. Y.Z. and J.M. supervised the entire project and made final revisions to the paper.

Competing interests

The authors declare no competing interests.

Additional information

Extended data is available for this paper at <https://doi.org/10.1038/s42256-025-01126-3>.

Supplementary information The online version contains supplementary material available at <https://doi.org/10.1038/s42256-025-01126-3>.

Correspondence and requests for materials should be addressed to Shing-Tung Yau, Yuan Zhou or Jianzhu Ma.

Peer review information *Nature Machine Intelligence* thanks Parshin Shojaee and the other, anonymous, reviewer(s) for their contribution to the peer review of this work.

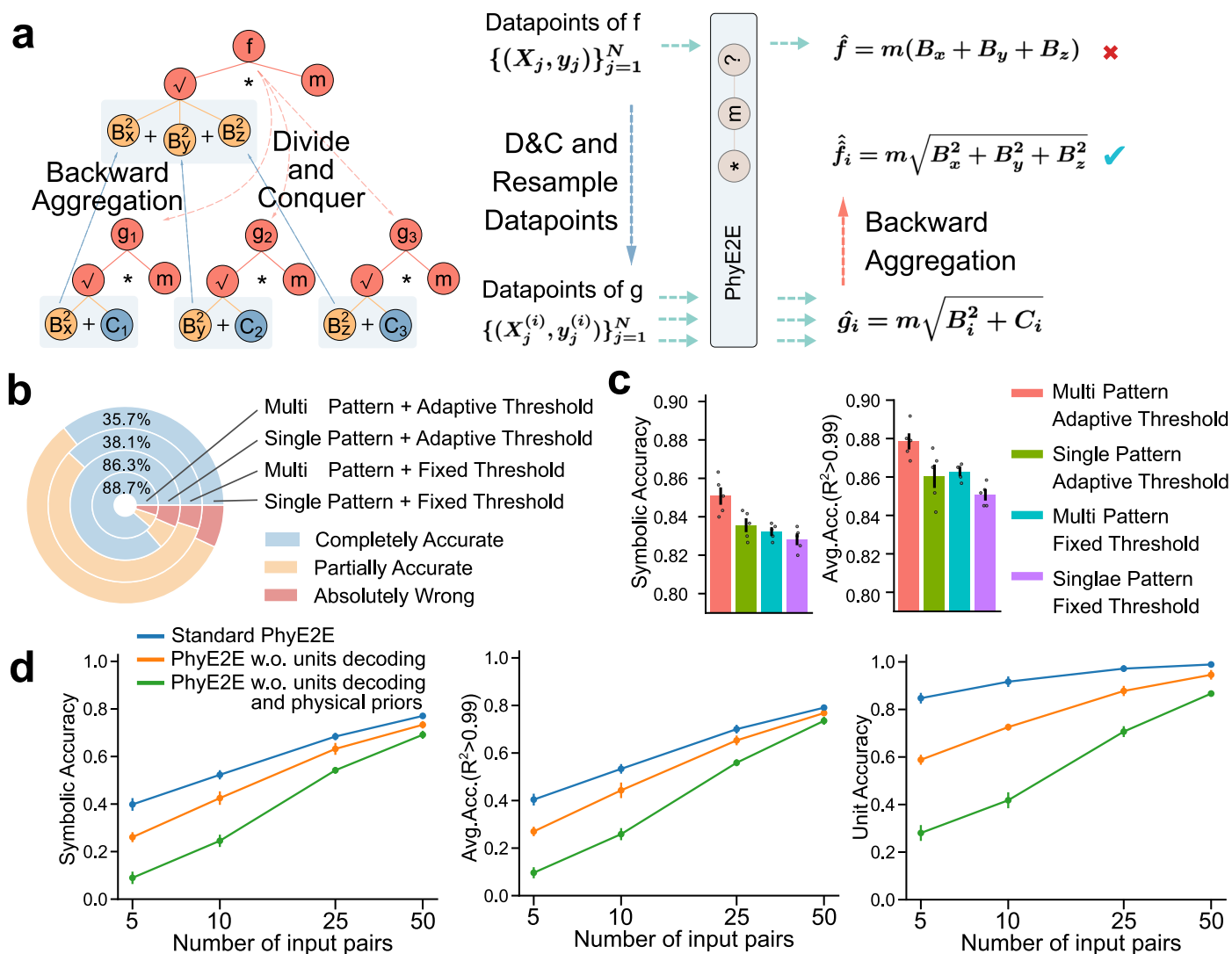
Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

© The Author(s), under exclusive licence to Springer Nature Limited 2025

¹Qiuzhen College, Tsinghua University, Beijing, China. ²Institute for Artificial Intelligence, Peking University, Beijing, China. ³School of Earth and Space Sciences, Peking University, Beijing, China. ⁴Max Planck Institute for Solar System Research, Göttingen, Germany. ⁵Institute of Space Sciences, Shandong University, Weihai, China. ⁶Yau Mathematical Sciences Center, Tsinghua University, Beijing, China. ⁷Beijing Institute of Mathematical Sciences and Applications, Beijing, China. ⁸Department of Mathematical Sciences, Tsinghua University, Beijing, China. ⁹Department of Electronic Engineering, Tsinghua University, Beijing, China. ¹⁰Institute for AI Industry Research, Tsinghua University, Beijing, China. ¹¹These authors contributed equally: Jie Ying, Haowei Lin, Chao Yue. ✉ e-mail: styau@tsinghua.edu.cn; yuan-zhou@tsinghua.edu.cn; majianzhu@tsinghua.edu.cn

**Extended Data Fig. 1 | Performance on Synthetic and AI Feynman datasets.**

a, An example of D&C procedure, including Divide-and-Conquer and Backward Aggregation step. **b**, Decomposition accuracy for different D&C strategies.

c, Symbolic accuracy and average accuracy of $R^2 > 0.99$ under different D&C

strategies. Data are presented as mean values $\pm$ SEM ($n = 5$ individual trials for each configuration). **d**, The accuracy performance on low data cases with different physical priors incorporated into PhyE2E. Data are presented as mean values $\pm$ SEM ($n = 5$ individual trials for each configuration).